

readme

CCF Lab 3 — Sandboxing & Malware Analysis

Name of report: Sandboxing_LAB_3_Nikita_Niakhai

Course: Forensics

Performed by *Nikita Niakhai*

Date of submission: 2026-04-28

Host OS: Ubuntu Linux 6.8.0-110-generic

Hypervisor: QEMU/KVM via virt-manager

Sandbox: CAPEv2

Analysis VM: Windows 10 x64 (win10.qcow2)

Environment Overview

The lab runs on a single Ubuntu desktop (SNE lab PC, which also has second-primary user `victor`) as a `root` and `nikita` users. The analysis VM is isolated on a host-only bridge network. CAPE controls the VM over that same network.

For VM manager `Virtual Manager` was chosen.

So, please let `victor` username for root@victor do not bother you, there are still screenshots with `nikita` user ;)

BTW, I can show you this PC with the lab ENV.

Claude PRO was used to analyze logs and guide through complex situations, concerning security breaches.

Architecture

```
Host - 192.168.100.1 (virbr0)
├─ win10 VM - 192.168.100.10 (static)
│   └─ agent.py listening on :8000
│       └─ reports back to resultserver at 192.168.100.1:2042
```

Task 1 — Preparation

1.1 Sandbox Selection

CAPEv2 (Config And Payload Extraction) was chosen as the sandboxing solution.

Key reasons:

- Active fork of Cuckoo Sandbox with regular updates
- Built-in PE unpacking and payload extraction engine — CAPE can unpack packed samples and automatically submit extracted payloads for re-analysis
- Native .NET and Themida support
- YARA and community signature matching out of the box
- Self-hosted, no sample privacy concerns

Installation:

```
git clone https://github.com/kevoreilly/CAPEv2 /opt/CAPEv2
cd /opt/CAPEv2
sudo -u cape poetry install
```

CAPE runs as a set of independent services:

Service	Role
cape.service	Core scheduler, VM controller, hook injector
cape-web.service	Web dashboard on port 8000
cape-processor.service	Post-analysis report generation
cape-rooter.service	Per-analysis network routing
mongodb.service	Report storage
postgresql@18-main.service	Task and metadata database

1.2 VM Setup

The analysis VM runs Windows 10 x64 under QEMU/KVM managed by virt-manager.

Parameter	Value
OS	Windows 10 x64
CPU	3 vCPUs, host-passthrough mode
RAM	6 GB
Disk	<code>/var/lib/libvirt/images/win10.qcow2</code>
Machine type	pc-q35-6.2
Disk bus	SATA

1.3 Network Configuration

The VM connects to `virbr0` — a libvirt-managed bridge at `192.168.100.1/24`. The VM has a static IP of `192.168.100.10`.

The libvirt default network has NAT forwarding enabled. This means:

- The VM can reach the internet through the host — useful for capturing C2 callbacks in network logs
- For destructive samples like ransomware, NAT should be disabled to keep the VM fully isolated

```
win10 on QEMU/KVM
Virtual Machine View Send Key
File Edit View Bookmarks Plugins Settings Help
.: sudo -i - Konsole <2>
inet6 ::1/128 scope host
    valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 6c:4b:90:14:95:23 brd ff:ff:ff:ff:ff:ff
    inet 10.92.0.142/22 brd 10.92.3.255 scope global dynamic noprefixroute enp1s0
        valid_lft 84929sec preferred_lft 84929sec
    inet6 fe80::b6f5:c79:c4b0:acd2/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp2s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 18:d6:c7:00:e8:ad brd ff:ff:ff:ff:ff:ff
4: virbr2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 52:54:00:74:82:29 brd ff:ff:ff:ff:ff:ff
    inet 172.16.0.1/24 brd 172.16.0.255 scope global virbr2
        valid_lft forever preferred_lft forever
5: virbr0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 52:54:00:b2:b8:e6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.1/24 brd 192.168.100.255 scope global virbr0
        valid_lft forever preferred_lft forever
6: virbr1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:60:00:00 brd 52:54:00:60:00:00
    inet 10.0.0.1/24 brd 10.0.0.255 scope global virbr1
        valid_lft forever preferred_lft forever
7: br-02f397e24387: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 02:00:00:00:00:00 brd 02:00:00:00:00:00
    inet 10.0.0.1/24 brd 10.0.0.255 scope global br-02f397e24387
        valid_lft forever preferred_lft forever
```

Figure 1.1 — virbr0 host-only adapter visible via `ip a`

Figure 1.1 — virbr0 adapter at 192.168.100.1/24 on the host

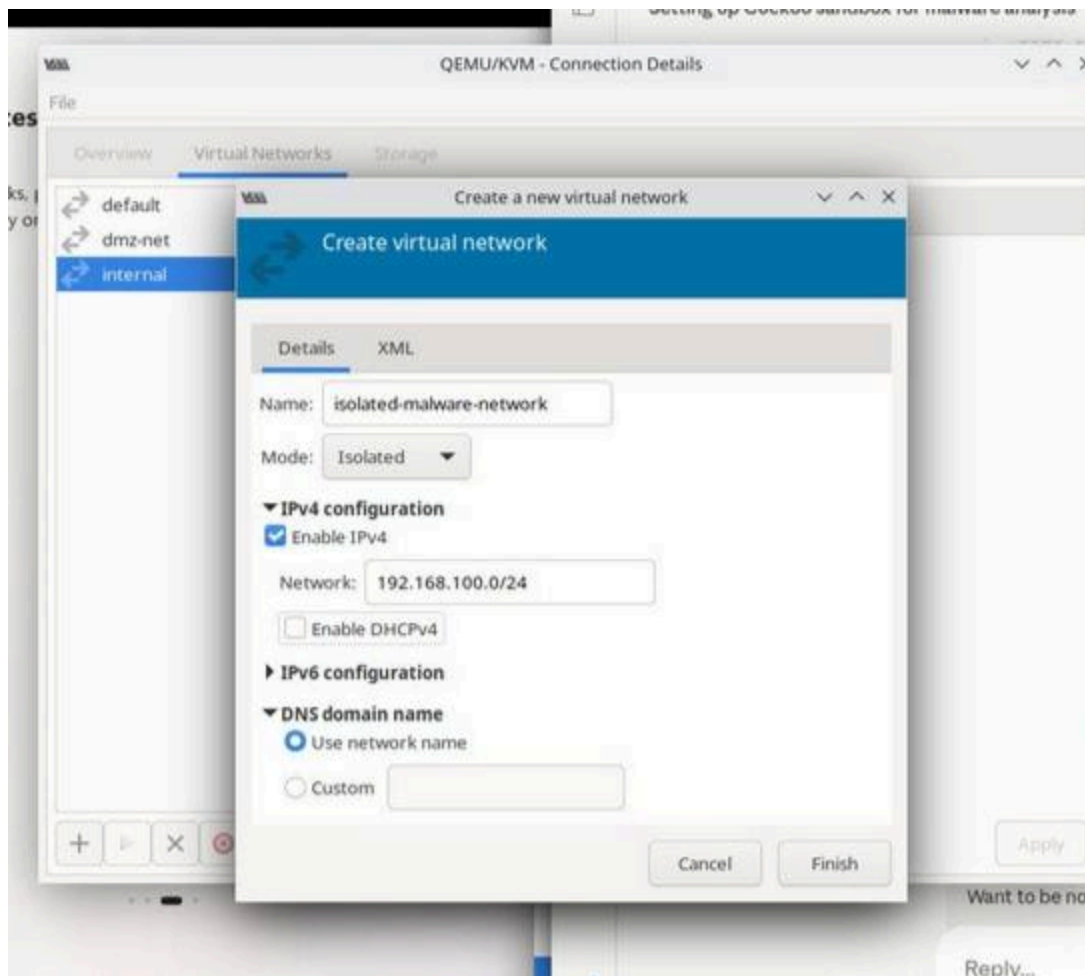


Figure 1.2 — NAT network configuration in virt-manager

Figure 1.2 — libvirt NAT network configuration

CAPE configuration pointing at the VM:

```
# /opt/CAPEv2/conf/kvm.conf
[kvm]
machines= win10
interface= virbr0

[win10]
label= win10
platform= windows
ip= 192.168.100.10
snapshot= snapshot1
interface= virbr0
resultserver_ip= 192.168.100.1
resultserver_port=2042
```

```
arch= x64
tags= win10
```

```
# /opt/CAPEv2/conf/cuckoo.conf
[resultserver]
ip= 192.168.100.1
port=2042
```

1.4 CAPE Agent Setup

The CAPE agent (`agent.py`) runs inside the Windows VM and acts as CAPE's hands inside the guest. It receives hook DLLs from CAPE, executes samples, and sends behavioral data back to the result server.

Requirements inside the VM:

- Must run as Administrator — hook injection requires elevated privileges
- Windows Firewall must be disabled — otherwise port 8000 is blocked

Files were transferred to the VM over the `virbr0` network using a Python HTTP server on the host:

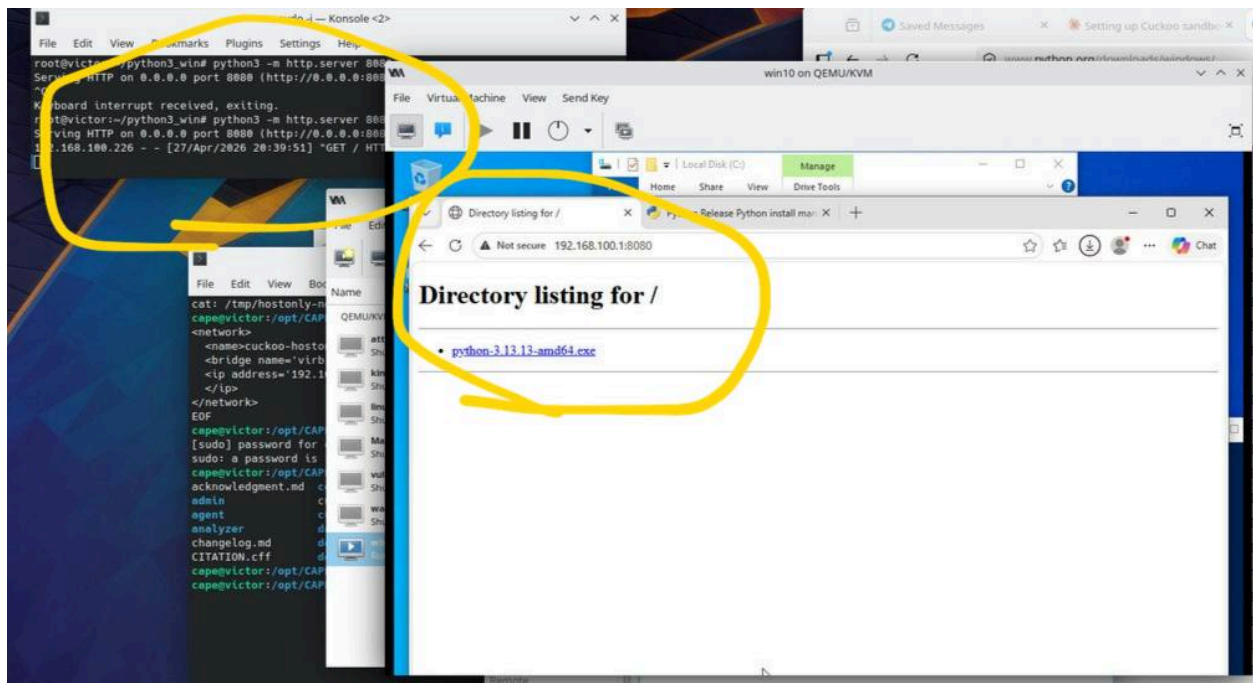


Figure 1.3 — Files transferred to VM via Python HTTP server

Figure 1.3 — `python3 -m http.server` used to deliver `agent.py` to the VM over `virbr0`

Inside the VM (admin PowerShell):

```
netsh advfirewall set allprofiles state off
python C:\agent.py
```

Verify connectivity from host:

```
nc -z -w2 192.168.100.10 8000 && echo "AGENT OK"
```

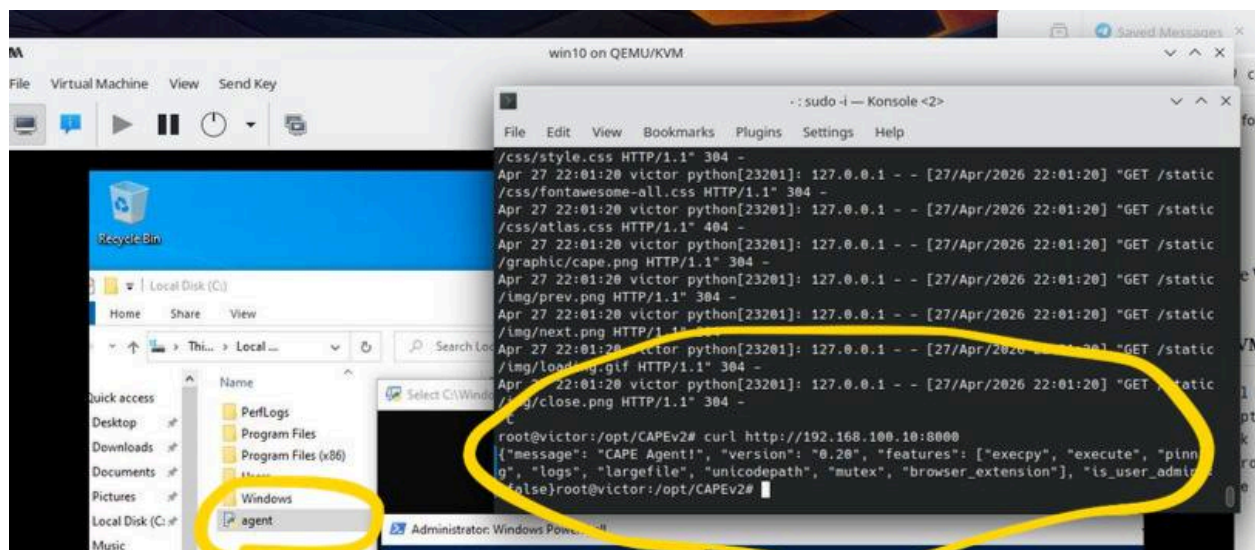


Figure 1.4 — `agent.py` running inside the Windows VM

Figure 1.4 — `agent.py` listening on port 8000 in admin PowerShell

1.5 CAPE Dashboard

Once the services are running, the web dashboard is accessible from the host browser.

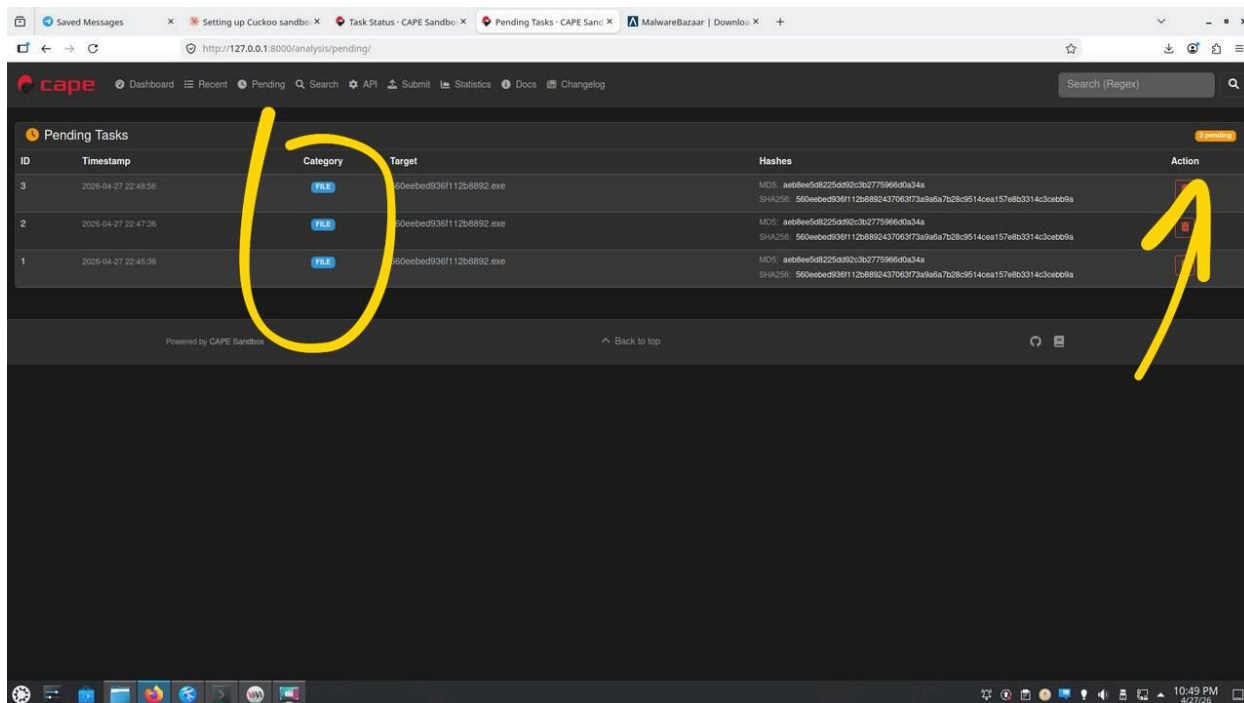


Figure 1.5 — CAPE web dashboard running

Figure 1.5 — CAPE dashboard showing active task queue

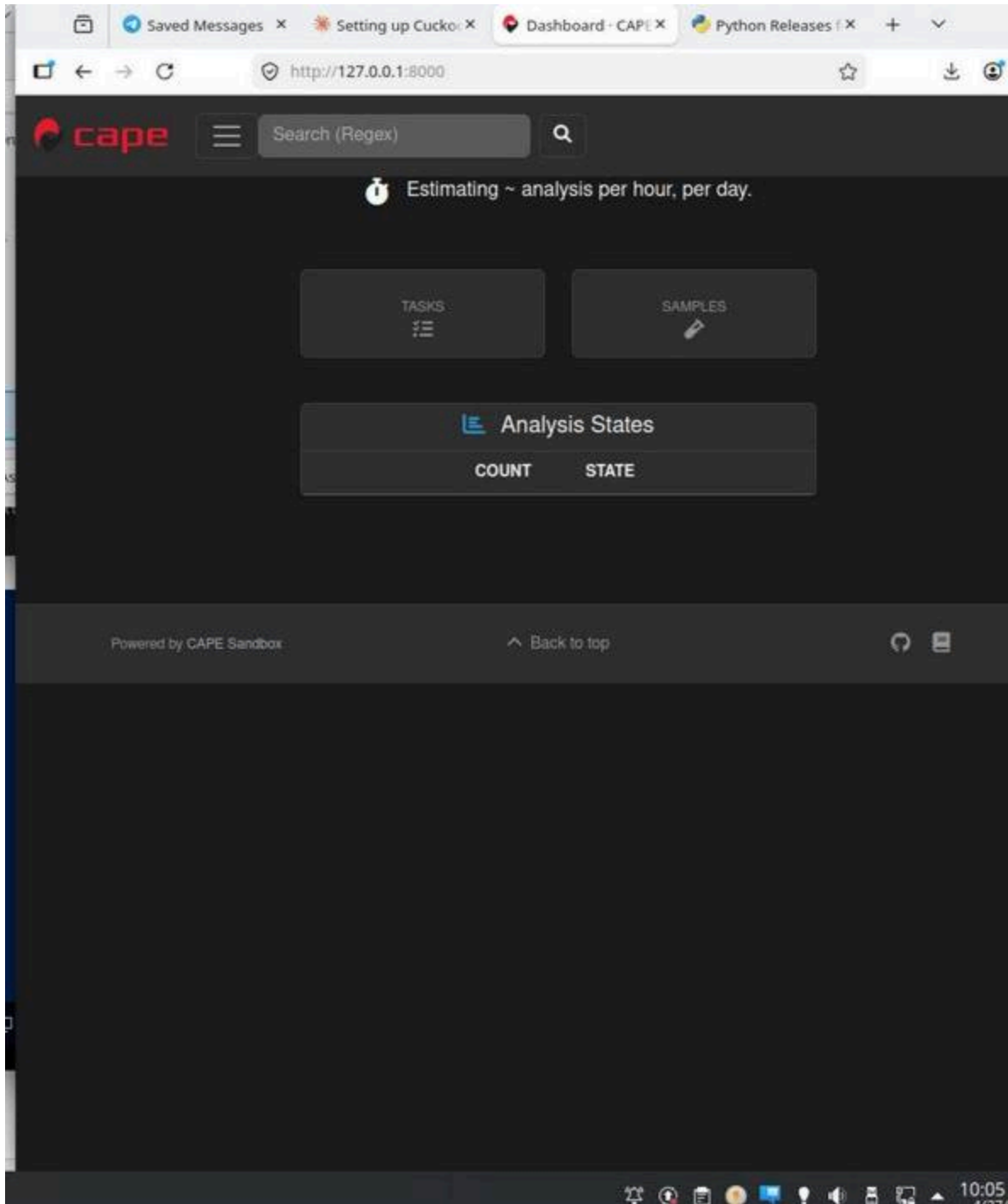


Figure 1.6 — CAPE dashboard accessible from host

Figure 1.6 — Dashboard opened from the host machine, confirming network connectivity

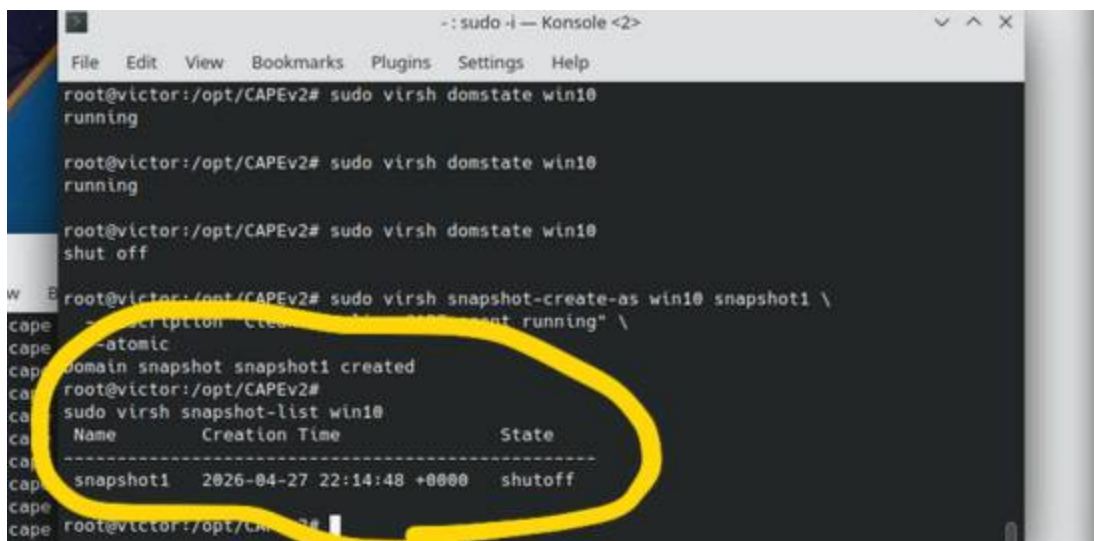
1.6 Snapshot

CAPE requires a libvirt snapshot in `running` state — the VM must be captured with the agent already active inside. CAPE restores this snapshot between every analysis run to guarantee a clean environment.

```
virsh snapshot-create-as \
  --domain win10 \
  --name snapshot1 \
  --description "CAPE clean state with agent running" \
  --atomic
```

The `--atomic` flag is required because the disk uses a SATA bus — live snapshots are not supported on SATA. `--atomic` pauses the VM briefly, takes the snapshot, then resumes.

```
# Verify state = running (CAPE checks this on startup)
virsh snapshot-info win10 snapshot1
```



```
root@victor:/opt/CAPEv2# sudo virsh domstate win10
running

root@victor:/opt/CAPEv2# sudo virsh domstate win10
running

root@victor:/opt/CAPEv2# sudo virsh domstate win10
shut off

root@victor:/opt/CAPEv2# sudo virsh snapshot-create-as win10 snapshot1 \
  --description "CAPE clean state with agent running" \
  --atomic
domain snapshot snapshot1 created
root@victor:/opt/CAPEv2#
root@victor:/opt/CAPEv2# sudo virsh snapshot-list win10
+-----+
| Name           | Creation Time           | State   |
+-----+-----+-----+
| snapshot1      | 2026-04-27 22:14:48 +0000 | shutoff |
+-----+-----+-----+
```

Figure 1.7 — snapshot1 created successfully

Figure 1.7 — virsh confirms snapshot1 in running state

```
root@victor:~# systemctl status cape.service
● cape.service - CAPE
   Loaded: loaded (/lib/systemd/system/cape.service; enabled; vendor preset: enabled)
   Active: activating (auto-restart) (Result: exit-code) since Mon 2026-04-27 22:36:
   Docs: https://github.com/kevoreilly/CAPEv2
   Process: 35294 ExecStart=/opt/CAPEv2/.cache/pypoetry/virtualenvs/capev2-t2x27zRb-p
   Main PID: 35294 (code=exited, status=1/FAILURE)
   CPU: 5.380s
```

Figure 1.8 — cape.service active and stable

Figure 1.8 — `systemctl status cape.service` showing active with no errors

1.7 Issues Encountered During Setup

Issue	Cause	Fix
<code>cape.service</code> crashed: "snapshot not found"	No snapshot existed yet	Created <code>snapshot1</code> with <code>virsh snapshot-create-as</code>
<code>cape.service</code> crashed: resultserver IP mismatch	Config had <code>ip = 192.168.1.1</code> instead of <code>192.168.100.1</code>	Updated <code>cuckoo.conf</code> resultserver IP
Snapshot state was <code>shutoff</code> instead of <code>running</code>	Snapshot taken without <code>--atomic</code> while VM was paused	Re-created snapshot using <code>--atomic</code> flag
Agent not running as admin	Launched from a regular terminal	Moved to admin PowerShell — hooks require elevated privileges

Task 2 — Malware Samples

Two samples were analyzed. The second was discovered automatically by CAPE during analysis of the first.

Sample 1 — Outer Dropper/Loader

Downloaded from **MalwareBazaar**.

Signature, tag or malware family.

Browse Database

See search syntax see below, example: tag:TrickBot

Search Syntax ?

Search: formbook

Date (UTC)	SHA256 hash	Type	Signature	Tags	Reporter	DL
2026-04-27 07:55	560eebed936f112b8892...	exe	Formbook	exe FormBook	adrian_luca	
2026-04-27 07:55	ae7859250a08f3a5b048...	exe	Formbook	exe FormBook	adrian_luca	
2026-04-27 04:54	eb11e9e446bb14380d2...	exe	Formbook	exe FormBook	threatcat_ch	
2026-04-27 04:44	45fc07f4eca491881b53...	exe	Formbook	exe FormBook	threatcat_ch	
2026-04-27 03:31	e1134e75fde09057073a...	ps1	Formbook	FormBook ps1	BastianHein	
2026-04-27 03:00	759066f9c70d498ba50b...	vbe	Formbook	FormBook vbe	FXOLabs	

Showing 1 to 6 of 6 entries (filtered from 1,000 total entries)

Previous 1 Next

[Terms and Conditions](#) |
 [Terms of Use](#) |
 [Privacy Policy](#) |
 [Cookie Policy](#)

Figure 2.1 — Downloading sample from MalwareBazaar

Figure 2.1 — MalwareBazaar download page for the outer dropper

MalwareBazaar Database

This page let you download the following malware sample: **SHA256**
560eebed936f112b8892437063f73a9a6a7b28c9514cea157e8b3314c3cebb9a

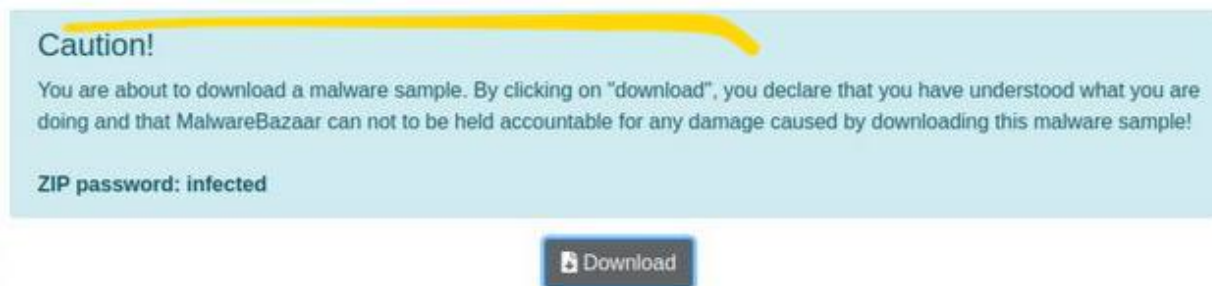


Figure 2.2 — Archive downloaded to host

Figure 2.2 — Zip archive confirmed present in Downloads folder

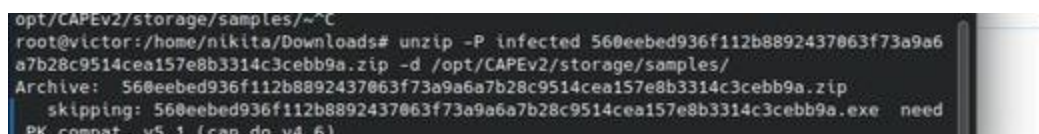


Figure 2.3 — Unzipping sample to working directory

Figure 2.3 — Extracting with password **infected** (MalwareBazaar standard)

Field	Value
SHA256	560eebed...
Type	PE32 .NET assembly (x86), GUI
Role	Outer dropper — unpacks and launches the inner payload

Sample 2 — Nested Payload (Extracted by CAPE)

CAPE's extraction engine found a second PE binary packed inside Sample 1 during the first analysis run. It was automatically stored and submitted for separate analysis.

Field	Value
SHA256	0ba17c89e787fee76c372b36e8db40ffc31f96420ae882a78f603e5c5e228005

Field	Value
MD5	4b4b5334b25f14402d7d236e2c49713d
Type	PE32 executable (GUI) Intel 80386, Mono/.NET assembly
Size	1,107,456 bytes
Packer	Themida (commercial protector/DRM)
Origin	Extracted from Sample 1 by CAPE during Task 1 analysis

This nested PE is the main subject of analysis for the rest of the lab.

Task 3 — Sandbox Analysis

3.1 Submitting Sample 1 — Behavior Capture

Sample 1 was submitted to CAPE using the CLI tool:

```
cd /opt/CAPEv2
sudo -u cape .cache/pypoetry/virtualenvs/capev2-t2x27zRb-py3.
10/bin/python \
  utils/submit.py /opt/CAPEv2/storage/binaries/560eebed... \
  --timeout 240 --enforce-timeout
```

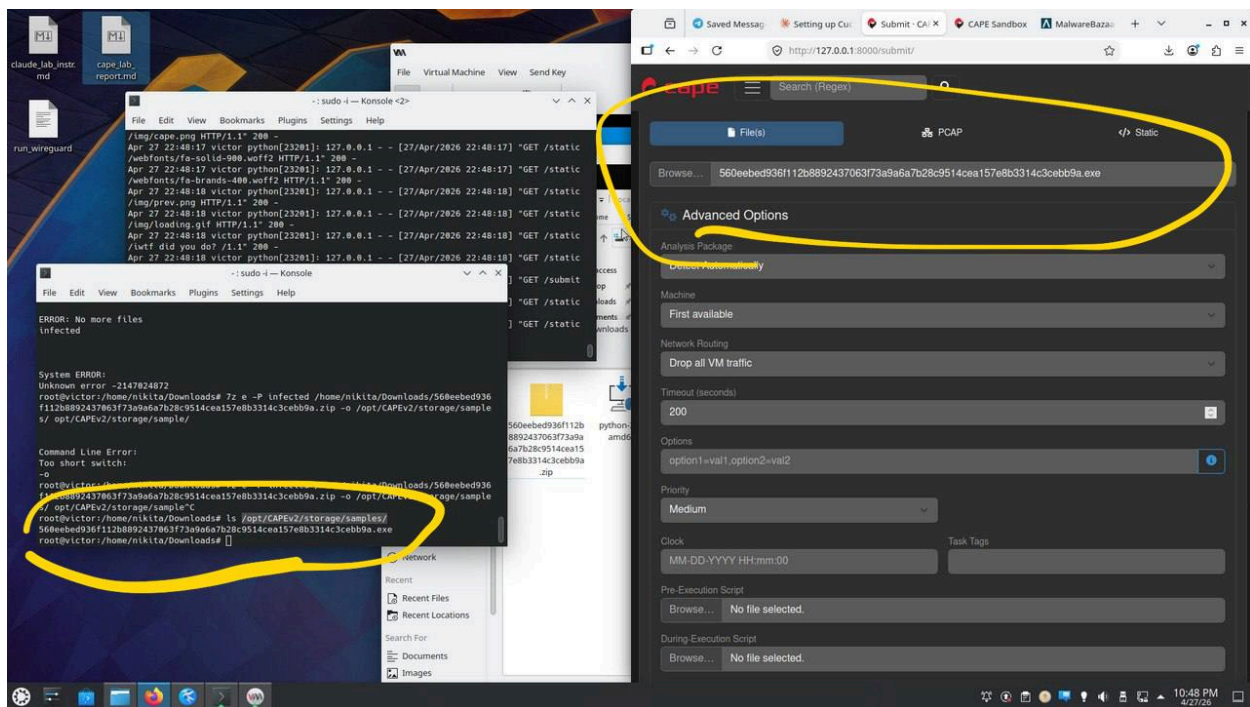


Figure 3.1 — Sample submitted for analysis

Figure 3.1 — submit.py call confirming sample queued as task 1

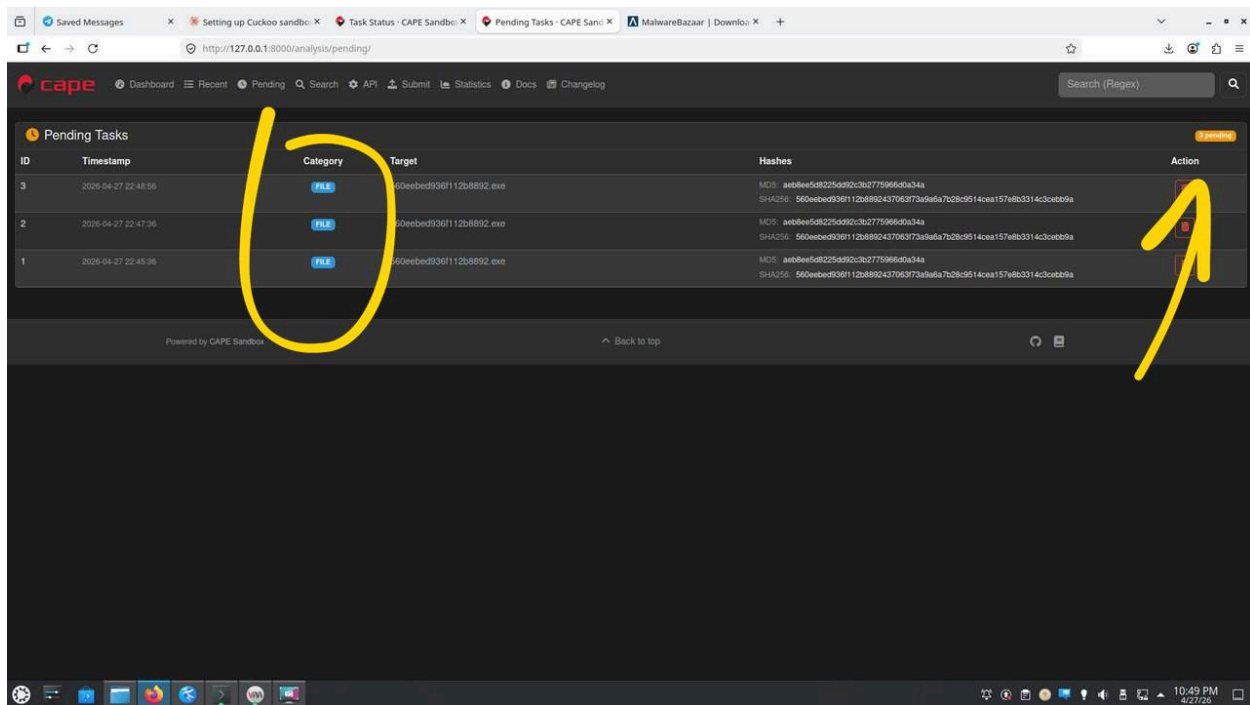


Figure 3.2 — Tasks 1, 2, 3 pending in CAPE dashboard

Figure 3.2 — Three tasks queued for the outer dropper with different option combinations

```
y_manager] INFO: Using MachineryManager[kvm] with max_machines_count=10
Apr 27 22:53:34 victor python[39694]: 2026-04-27 22:53:34,430 [lib.cuckoo.core.scheduler] INFO: Creating scheduler with max_analysis_count=unlimited
Apr 27 22:53:37 victor python[39694]: 2026-04-27 22:53:37,722 [lib.cuckoo.core.machinery_manager] INFO: Loaded 1 machine
Apr 27 22:53:38 victor python[39694]: 2026-04-27 22:53:38,444 [lib.cuckoo.core.machinery_manager] INFO: max_vmstartup_count for BoundedSemaphore = 5
Apr 27 22:53:38 victor python[39694]: 2026-04-27 22:53:38,482 [lib.cuckoo.core.scheduler] INFO: Waiting for analysis tasks
Apr 27 22:53:38 victor python[39694]: 2026-04-27 22:53:38,549 [lib.cuckoo.core.machinery_manager] INFO: Task #1: found useable machine win10 (arch=x64, platform=windows)
Apr 27 22:53:38 victor python[39694]: 2026-04-27 22:53:38,550 [lib.cuckoo.core.scheduler] INFO: Task #1: Processing task
Apr 27 22:53:38 victor python[39694]: 2026-04-27 22:53:38,749 [lib.cuckoo.core.analysis_manager] INFO: Starting analysis of FILE '/tmp/cuckoo-sflock/tmpd91g7dfu/560eebed936f112b8892.exe'
```

Figure 3.3 — journalctl showing CAPE processing the sample

Figure 3.3 — Live cape.service log: VM snapshot restored, sample injected, monitoring active

Results — Sample 1

Field	Value
Malscore	2.0 / 10
Processes captured	0
Network activity	None
Payloads extracted	1 (Sample 2)

Signatures triggered:

Signature	Severity	Meaning
packer_entropy	Medium	High-entropy sections — binary is packed or encrypted
pe_compile_timestomping	High	PE timestamp is fake — deliberate anti-forensics

The outer dropper executed, unpacked its payload into memory, and exited — all before CAPE’s hook layer could observe any behavior. The only useful output was the extracted nested PE.

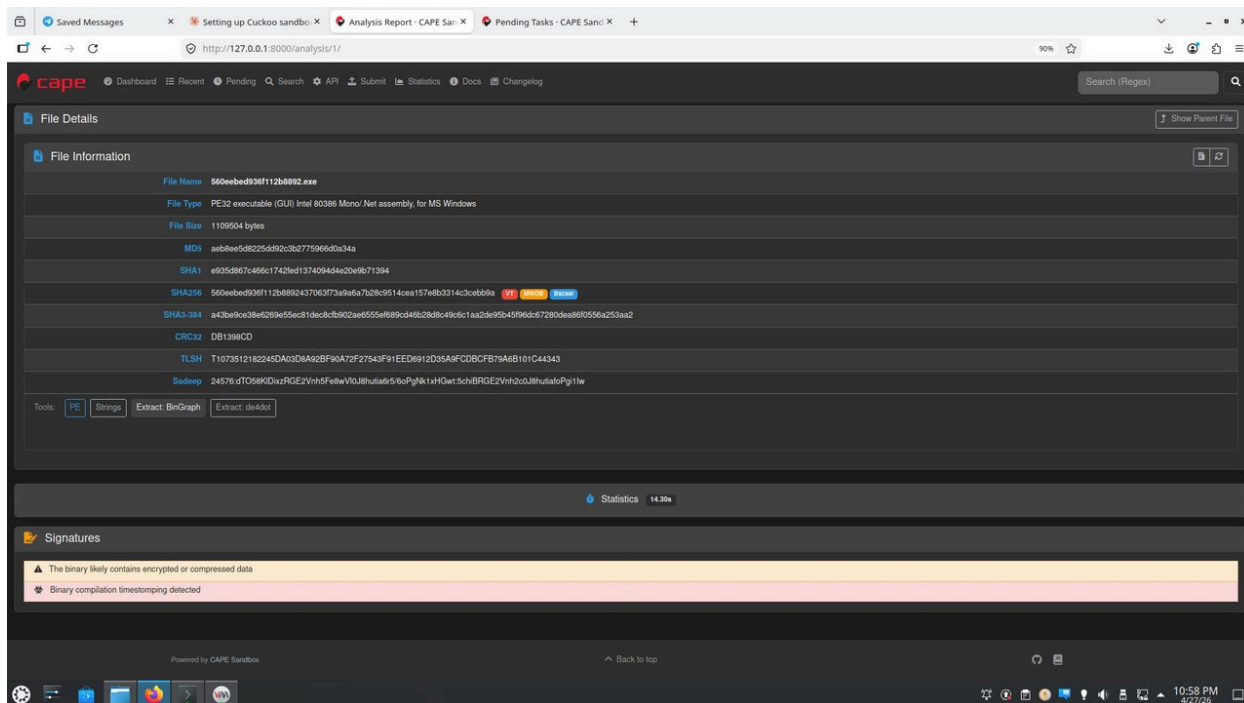


Figure 3.4 — Tasks 1–3 completed in CAPE dashboard

Figure 3.4 — All three initial tasks completed. Malscore 2.0 with zero behavioral data.

3.2 Submitting Sample 2 — Evasion and Defeat Attempts

CAPE found the nested PE and flagged it for separate analysis. It was sent as task 4.

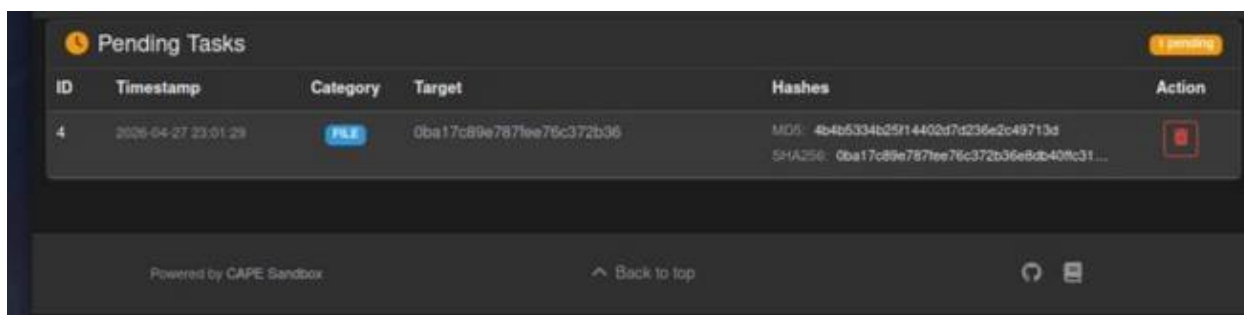


Figure 3.5 — Nested payload extracted and submitted as task 4

Figure 3.5 — CAPE reports payload found; submitted automatically as task 4

Sample 2 is packed with **Themida** — a commercial protector used both for software DRM and by malware authors. Themida performs a series of environment checks before decompressing and running the real code. If it detects a sandbox or debugger, it exits silently.

Evasion techniques detected on Sample 2:

Signature	Technique
<code>packer_themida</code>	Themida protector wrapper
<code>antisandbox_cuckoocrash</code>	Detects CAPE/Cuckoo by name or artifacts
<code>antidebug_guardpages</code>	Memory guard pages to detect debugger
<code>antidebug_outputdebugstring</code>	Timing check via OutputDebugString
<code>antidebug_windows</code>	IsDebuggerPresent / NtQueryInformationProcess
<code>mouse_movement_detect</code>	Checks for real mouse input
<code>antisandbox_foregroundwindows</code>	Checks if a real window is in foreground
<code>antisandbox_sleep</code>	Long sleep calls to outlast sandbox timeout
<code>antisandbox_script_timer</code>	Detects accelerated timers
<code>antiav_servicestop</code>	Tries to stop AV/EDR services
<code>stealth_network</code>	Hides network activity
<code>disable_driver_via_blocklist</code>	Blocks security driver loading
<code>pe_compile_timestomping</code>	Fake compile timestamp

13 evasion techniques total — this is an unusually high count, consistent with Themida's DRM-grade protection.

3.3 Defeating Sandbox Evasion — Four Attempts

Attempt 1 — Default CAPE hooks (Task 4)

Standard submission with full hook injection. Themida detected the hook footprint immediately and exited.

Result: Malscore 2.0, 0 processes, 0 network.

Attempt 2 — Relaxed options (Task 5)

Resubmitted with options designed to look more like a real system:

```
free=yes    - disables CAPE hooks entirely
human=1     - simulates mouse movement
sleep=1     - patches sleep calls so Themida can't outlast timeout
clock=...   - fakes system date
```

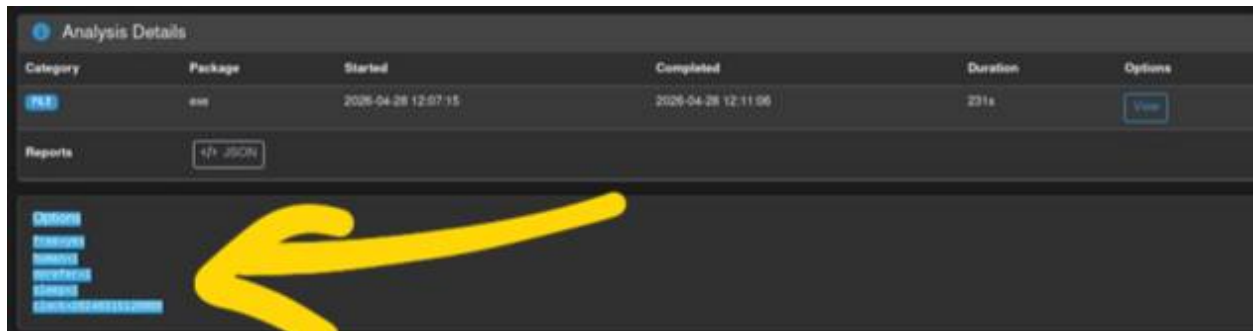


Figure 3.7 — Submission options for nested payload analysis

Figure 3.7 — CAPE submission form with evasion-defeat options applied

`free=yes` removes the hook footprint Themida was detecting. But the result was the same — Themida still exited without executing.

Result: Malscore 2.0, 0 processes, 0 network.

Conclusion: hook detection was not the only problem. Themida was also detecting the QEMU virtual machine itself.

Attempts 3 and 4 — VM Hardware Hardening (Tasks 6 and 7)

Themida detects QEMU through four main vectors:

- **CPUID hypervisor bit** — KVM sets a flag in the CPU ID response that signals a hypervisor is present
- **SMBIOS strings** — QEMU puts "QEMU" in the BIOS/motherboard info readable by any application
- **MAC address OUI** — QEMU uses the OUI prefix `52:54:00`, registered to QEMU
- **Disk serial** — Windows Device Manager shows the device as "QEMU HARDDISK"

The libvirt VM XML was modified to spoof all four:

```
<!-- Hide hypervisor flag from CPUID -->
<kvm><hidden state='on' /></kvm>
<cpu mode='host-passthrough' check='none' migratable='on'>
  <feature policy='disable' name='hypervisor' />
</cpu>

<!-- Spoof SMBIOS as HP EliteBook 840 G8 -->
<sysinfo type='smbios'>
  <bios>
    <entry name='vendor'>American Megatrends Inc.</entry>
    <entry name='version'>F.71</entry>
    <entry name='date'>09/14/2021</entry>
  </bios>
  <system>
    <entry name='manufacturer'>HP</entry>
    <entry name='product'>HP EliteBook 840 G8 Notebook PC</entry>
    <entry name='serial'>CND1471B3M</entry>
  </system>
</sysinfo>
<os>
  <smbios mode='sysinfo' />
</os>

<!-- Replace QEMU MAC OUI with a real Intel OUI -->
<mac address='00:21:cc:a4:3f:7e' />

<!-- Add a realistic Western Digital disk serial -->
<serial>WD-WCC4N2RKV8F3</serial>
```

Applied with:

```
virsh define /tmp/win10.xml
```

The VM was rebooted to apply the SMBIOS changes.

```

-: sudo -i -- Konsole <2>
File Edit View Bookmarks Plugins Settings Help
nikita@victor:~$ ^C nc -z -w2 192.168.100.10 8000 && echo "AGENT OK" || echo "AGENT UNREACHABLE"
^C
nikita@victor:~$ nc -z -w2 192.168.100.10 8000 && echo "AGENT OK" || echo "AGENT UNREACHABLE"
AGENT OK
nikita@victor:~$ sudo -i
[sudo] password for nikita:
root@victor:~# virsh snapshot-list win10 2>51; virsh domstate win10 2>51
Name              Creation Time      State
-----
snapshot1         2026-04-28 11:57:32 +0000    running
paused
root@victor:~#

```

Figure 3.6 — Snapshot recreated after VM hardening

Figure 3.6 — *virsh snapshot-list* confirming *snapshot1* in running state after VM reconfiguration

Tasks 6 and 7 were submitted against the hardened VM.

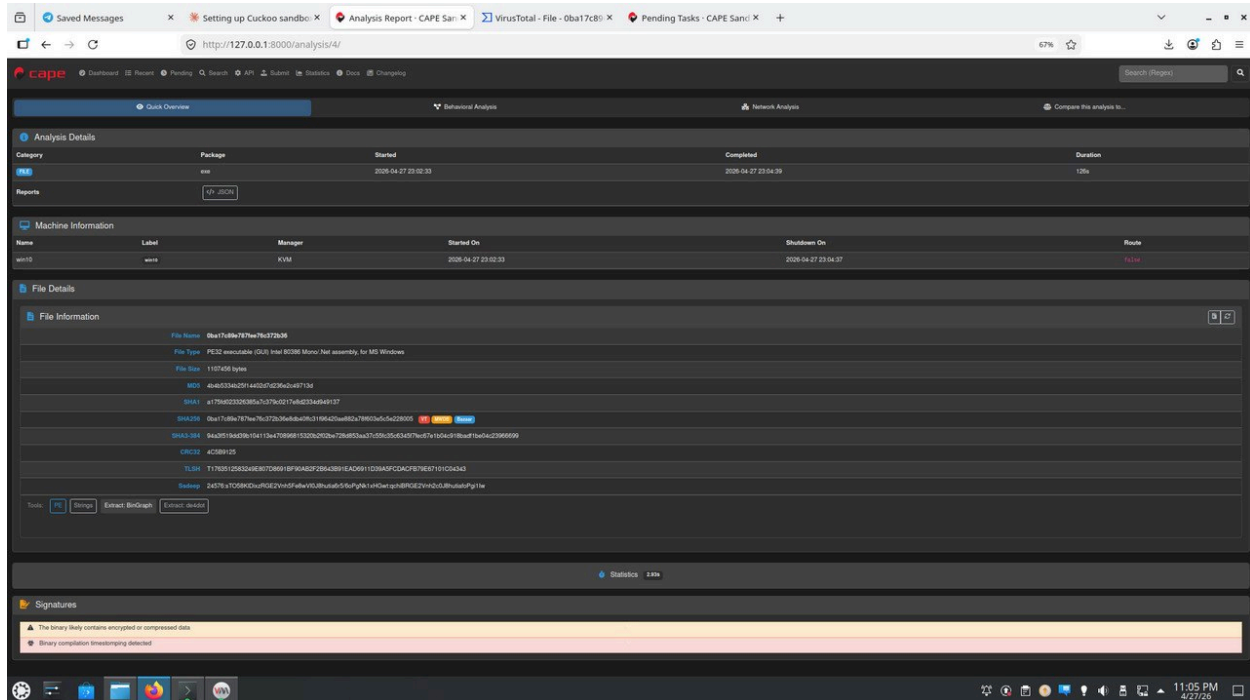


Figure 3.8 — Task 4 result: Malscore 2.0, 0 processes, 0 network — same as all previous runs

Figure 3.9 — Both tasks completed with zero behavioral data despite VM hardening

22

After the four vectors were addressed, Themida still detected the environment. Remaining detection points:

- **RDTSCTiming** — measures clock cycles between CPUID instructions. VM overhead is measurable even with host-passthrough CPU, because the hypervisor still intercepts some instructions
- **Registry artifacts** — QEMU/VirtIO driver keys are still present under `HKLM\SYSTEM\CurrentControlSet\Enum\`
- **SATA controller name** — Windows Device Manager still shows the QEMU SATA controller model name
- **Process scanning** — Themida may check for CAPE's monitoring processes or the agent itself

Summary of All Four Attempts

Task	Options	Result
4	Default hooks, admin agent	0 processes, 0 network
5	free=yes, human=1, sleep=1	0 processes, 0 network
6	Hardened VM (SMBIOS + MAC + KVM hidden)	0 processes, 0 network
7	Same hardened VM, second run	0 processes, 0 network

Conclusion: Themida's anti-VM layer is too deep for automated sandbox analysis on QEMU/KVM with the available options. Dynamic analysis failed completely. Static analysis is the only remaining approach.

3.4 Memory Dump and Volatility Analysis

Since the sample evaded all sandbox runs, a memory dump of the Windows VM was taken in its post-analysis state. This captures the full Windows environment: processes, network connections, loaded modules, and any injected code.

Dump Acquisition

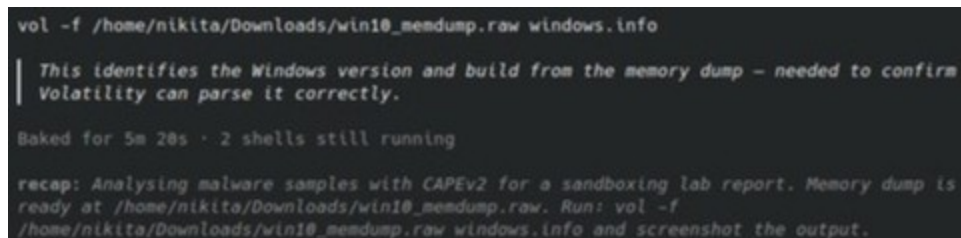
```
virsh dump win10 /home/nikita/Downloads/win10_memdump.raw --memory-only
```


Dump size: **6.1 GB** — matches the 6 GB RAM configured for the VM.

Step 1 — OS Identification (`windows.info`)

```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.info
```

The first step is always to confirm Volatility can correctly parse the dump and identify the OS version. This establishes the memory profile used by all subsequent plugins.



```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.info

This identifies the Windows version and build from the memory dump - needed to confirm
Volatility can parse it correctly.

Baked for 5m 28s - 2 shells still running

recap: Analysing malware samples with CAPEv2 for a sandboxing lab report. Memory dump is
ready at /home/nikita/Downloads/win10_memdump.raw. Run: vol -f
/home/nikita/Downloads/win10_memdump.raw windows.info and screenshot the output.
```

Figure 4.1 — windows.info output

Figure 4.1 — Volatility confirms Windows 10 x64 Build 19041, 3 CPUs, correct kernel base

Result: Windows 10 x64 Build 19041 confirmed. Dump is valid and parseable.

Step 2 — Process List (`windows.pslist`)

```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.pslist
```

Lists all processes in the kernel's EPROCESS doubly-linked list. This is the primary process list maintained by Windows. Shows PID, PPID, name, start time, and exit time.

Things to look for:

- Typosquatted names (`scvhost.exe` vs `svchost.exe`)
- Processes with no parent (orphans)
- Already-exited processes — malware that ran and terminated

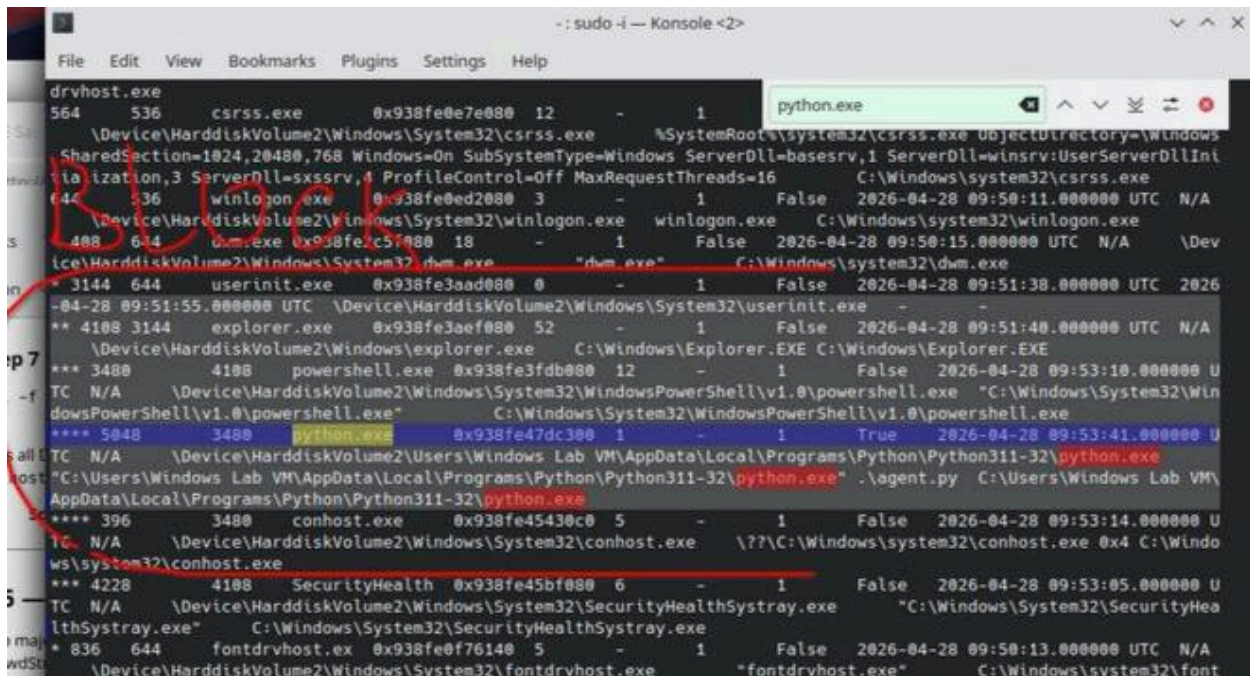


Figure 4.3 — pslist: agent chain part 1

Figure 4.3 — powershell.exe (PID 3480) and python.exe (PID 5048) visible in pslist

Notable processes:

PID	Name	PPID	Note
3480	powershell.exe	4108 (explorer)	Admin PowerShell used to launch the agent
5048	python.exe	3480	CAPE agent — <code>.\agent.py</code> , 32-bit (Wow64=True)
3128	MsMpEng.exe	688 (services)	Windows Defender — active, 27 threads
5420	svchost.exe	688 (services)	Exited at 09:54:27, 0 threads, blank cmdline

No typosquatted process names. No orphaned processes. The exited `svchost.exe` is normal — it is a transient host process for a service that completed.

Step 3 — Process Tree (`windows.psTree`)

```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.pstre  
e
```

Displays the same processes as a parent-child hierarchy. Useful for spotting suspicious spawn chains — for example, `winword.exe` launching `powershell.exe`, or `explorer.exe` spawning an unknown child.

A screenshot of the Windows Task Manager application, specifically the 'Processes' tab. The window title is '-: sudo -i -- Konsole <2>'. The menu bar includes 'File', 'Edit', 'View', 'Bookmarks', 'Plugins', 'Settings', and 'Help'. The process list shows various system and user processes. The 'explorer.exe' process, which is the Windows file explorer, is highlighted with a yellow arrow. Other visible processes include 'svchost.exe', 'ctfmon.exe', 'userinit.exe', 'NisSrv.exe', 'WmiPrvse.exe', 'StartMenuExe.exe', 'RuntimeBroker.exe', 'SearchIndexer.exe', 'SearchApp.exe', 'audiodg.exe', 'smartscreen.exe', and 'powershell.exe'. The 'explorer.exe' process has a PID of 3144 and is running under the 'explorer' user.

Figure 4.5 — Full process tree

Disabled	3060	688	svchost.exe	0x030fe40050e0	5	1	False	2026-04-28 00:53:23.000000 UTC	N/A
Disabled	5048	3480	python.exe	0x938fe47dc300	1	1	True	2026-04-28 09:53:41.000000 UTC	N/A
Disabled	5076	816	slui.exe	0x938fe409f080	5	1	False	2026-04-28 09:54:01.000000 UTC	N/A
Disabled	5420	688	svchost.exe	0x938fe3e1e080	0	0	False	2026-04-28 09:54:21.000000 UTC	2026-
04-28 09:54:27.000000 UTC	Disabled								
4692	816	WmiPrvSE.exe	0x938fe3e6a080	9	0	False	2026-04-28 09:54:30.000000 UTC	N/A	
Disabled									

Figure 4.6 — The agent launch chain clearly visible in the tree

Key chain:

```
explorer.exe (4108)
├─ powershell.exe (3480)    – admin PowerShell
│   └─ python.exe (5048)    – CAPE agent (.agent.py)
│       └─ conhost.exe (396) – console host
```

All processes trace back to legitimate system ancestors. No anomalous parent-child relationships.

Step 4 — Command Lines (`windows.cmdline`)

```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.cmdline
```

Extracts the full command line string for each process from the Process Environment Block (PEB). Reveals dropped file paths, encoded PowerShell payloads, and malware arguments.

3840	smartscreen.exe	C:\Windows\System32\smartscreen.exe -Embedding
4232	svchost.exe	C:\Windows\system32\svchost.exe -k netsvcs -p
4228	SecurityHealth	"C:\Windows\System32\SecurityHealthSystray.exe"
3468	SecurityHealth	C:\Windows\system32\SecurityHealthService.exe
3480	powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
396	conhost.exe	\\?C:\Windows\system32\conhost.exe 0x4
2672	SgrmBroker.exe	C:\Windows\system32\SgrmBroker.exe
6032	uhssvc.exe	"C:\Program Files\Microsoft Update Health Tools\unssvc.exe"
3880	svchost.exe	C:\Windows\System32\svchost.exe -k LocalServiceNetworkRestricted -p -s wscsvc
3060	svchost.exe	C:\Windows\system32\svchost.exe -k UninstallSvcGroup
5048	python.exe	"C:\Users\Windows Lab VM\AppData\Local\Programs\Python\Python311-32\python.exe" .agent.py
5076	slui.exe	C:\Windows\System32\slui.exe -Embedding
5420	svchost.exe	-
4692	WmiPrvSE.exe	C:\Windows\system32\wbem\wmiprvse.exe
488	ApplicationFrameHost.exe	C:\Windows\system32\ApplicationFrameHost.exe -Embedding
4768	WinStore.App.exe	"C:\Program Files\WindowsApps\Microsoft.WindowsStore_11910.1002.5.0_x64__8wekyb3d8bbwe\WinStore.App.exe" -ServerName:App.AppXc75wvmed5vhz4xyxxecvgdjhdkgzdza.mca

Figure 4.8 — cmdline: PID 5048 python.exe entry

Figure 4.8 — PID 5048: "...Python311-32\python.exe" .\agent.py — agent launched interactively

Notable entries:

PID	Process	Command
5048	python.exe	"...Python311-32\python.exe" .\agent.py
3976	slui.exe	SLUI.exe RuleId=3482d82e... Action=AutoActivate Trigger=NetworkAvailable
5420	svchost.exe	(blank — exited process, PEB cleared)

No encoded PowerShell. No dropped paths outside expected locations. The agent was run interactively from a PowerShell working directory — not installed as a scheduled task.

Step 5 — Network Connections (windows.netscan)

```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.netscan
```

Scans memory for TCP/UDP endpoint structures — active, closed, and listening sockets with remote IPs, ports, and owning process. The primary plugin for finding C2 communication.

```

File Edit View Bookmarks Plugins Settings Help
root@victor:~# vol -f /home/nikita/Downloads/win10_netdump.raw.windows.netscan
Volatility 3 Framework 2.27.0
Progress: 100.00
Offset Proto LocalAddr ForeignAddr ForeignPort State PID Owner Created
0x938fe0495310 TCPv4 0.0.0.0 49675 0.0.0.0 LISTENING 688 services.exe 2026-04-28 09:50:47.000000 UTC
0x938fe04955d0 TCPv4 0.0.0.0 49675 0.0.0.0 LISTENING 688 services.exe 2026-04-28 09:50:47.000000 UTC
0x938fe04955d0 TCPv6 :: 49675 :: 0 LISTENING 688 services.exe 2026-04-28 09:50:47.000000 UTC
0x938fe0495310 TCPv4 0.0.0.0 49666 0.0.0.0 LISTENING 1148 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe0495310 TCPv6 :: 49666 :: 0 LISTENING 1148 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe04955d0 TCPv4 0.0.0.0 49667 0.0.0.0 LISTENING 1712 spoolsv.exe 2026-04-28 09:50:20.000000 UTC
0x938fe04955d0 TCPv6 :: 49667 :: 0 LISTENING 1712 spoolsv.exe 2026-04-28 09:50:20.000000 UTC
0x938fe0495310 TCPv4 0.0.0.0 49668 0.0.0.0 LISTENING 1184 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe0495310 TCPv6 :: 49668 :: 0 LISTENING 1184 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe0495310 TCPv4 0.0.0.0 5040 0.0.0.0 LISTENING 1844 svchost.exe 2026-04-28 09:51:40.000000 UTC
0x938fe0495310 TCPv6 :: 5040 :: 0 LISTENING 1844 svchost.exe 2026-04-28 09:51:40.000000 UTC
0x938fe0495310 TCPv4 0.0.0.0 445 0.0.0.0 LISTENING 4 System 2026-04-28 09:50:31.000000 UTC
0x938fe0495310 TCPv6 :: 445 :: 0 LISTENING 4 System 2026-04-28 09:50:31.000000 UTC
0x938fe02931e0 UDPv4 192.168.100.10 137 * 0 4 System 2026-04-28 09:49:41.000000 UTC
0x938fe02931e0 UDPv6 :: 137 :: 0 LISTENING 4 System 2026-04-28 09:49:41.000000 UTC
0x938fe02931e0 TCPv4 192.168.100.10 139 0.0.0.0 LISTENING 788 lsass.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv6 :: 139 :: 0 LISTENING 788 lsass.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 49664 0.0.0.0 LISTENING 788 lsass.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv6 :: 49664 :: 0 LISTENING 788 lsass.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 135 0.0.0.0 LISTENING 944 svchost.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv6 :: 135 :: 0 LISTENING 944 svchost.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 49665 0.0.0.0 LISTENING 544 wininit.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv6 :: 49665 :: 0 LISTENING 544 wininit.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 49666 0.0.0.0 LISTENING 544 wininit.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv6 :: 49666 :: 0 LISTENING 544 wininit.exe 2026-04-28 09:50:14.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 49669 0.0.0.0 LISTENING 2636 svchost.exe 2026-04-28 09:50:36.000000 UTC
0x938fe02931e0 TCPv6 :: 49669 :: 0 LISTENING 2636 svchost.exe 2026-04-28 09:50:36.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 7680 0.0.0.0 LISTENING 5976 svchost.exe 2026-04-28 09:52:59.000000 UTC
0x938fe02931e0 TCPv6 :: 7680 :: 0 LISTENING 5976 svchost.exe 2026-04-28 09:52:59.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 8000 0.0.0.0 LISTENING 5048 python.exe 2026-04-28 09:53:49.000000 UTC
0x938fe02931e0 TCPv6 :: 8000 :: 0 LISTENING 5048 python.exe 2026-04-28 09:53:49.000000 UTC
0x938fe02931e0 TCPv4 0.0.0.0 49669 0.0.0.0 LISTENING 2636 svchost.exe 2026-04-28 09:50:36.000000 UTC
0x938fe02931e0 TCPv6 :: 49669 :: 0 LISTENING 2636 svchost.exe 2026-04-28 09:50:36.000000 UTC
0x938fe2df70 UDPv4 0.0.0.0 5353 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2df70 UDPv6 :: 5353 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 0 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 0 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 5355 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 5355 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 5355 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 5355 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 4580 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 4580 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 500 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 500 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 500 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 500 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 4580 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 4580 * 0 2628 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv4 0.0.0.0 0 * 0 2636 svchost.exe 2026-04-28 09:50:20.000000 UTC
0x938fe2ef3cd UDPv6 :: 0 * 0 2636 svchost.exe 2026-04-28 09:50:20.000000 UTC

```

Figure 4.9 — windows.netscan full output

Figure 4.9 — Full netscan output

```

File Edit View Bookmarks Plugins Settings Help
0000 UTC
0x938fe0a24730 TCPv6 :: 49669 :: 0 LISTENING python.exe
0000 UTC
0x938fe0a249f0 TCPv4 0.0.0.0 7680 0.0.0.0 LISTENING 5976 svchost.exe 2026-04-28 09:52:59.00
0000 UTC
0x938fe0a249f0 TCPv6 :: 7680 :: 0 LISTENING 5976 svchost.exe 2026-04-28 09:52:59.00
0000 UTC
0x938fe0a24b50 TCPv4 0.0.0.0 8000 0.0.0.0 LISTENING 5048 python.exe 2026-04-28 09:53:49.00
0000 UTC
0x938fe0a25390 TCPv4 0.0.0.0 49669 0.0.0.0 LISTENING 2636 svchost.exe 2026-04-28 09:50:36.00
0000 UTC
0x938fe2df70 UDPv4 0.0.0.0 5353 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 U
TC
0x938fe2ef3cd UDPv4 0.0.0.0 0 * 0 1808 svchost.exe 2026-04-28 09:50:20.000000 U

```

Figure 4.10 — netscan: agent LISTENING on :8000 (part 1)

Figure 4.10 — python.exe (5048) LISTENING on 0.0.0.0:8000 — agent socket

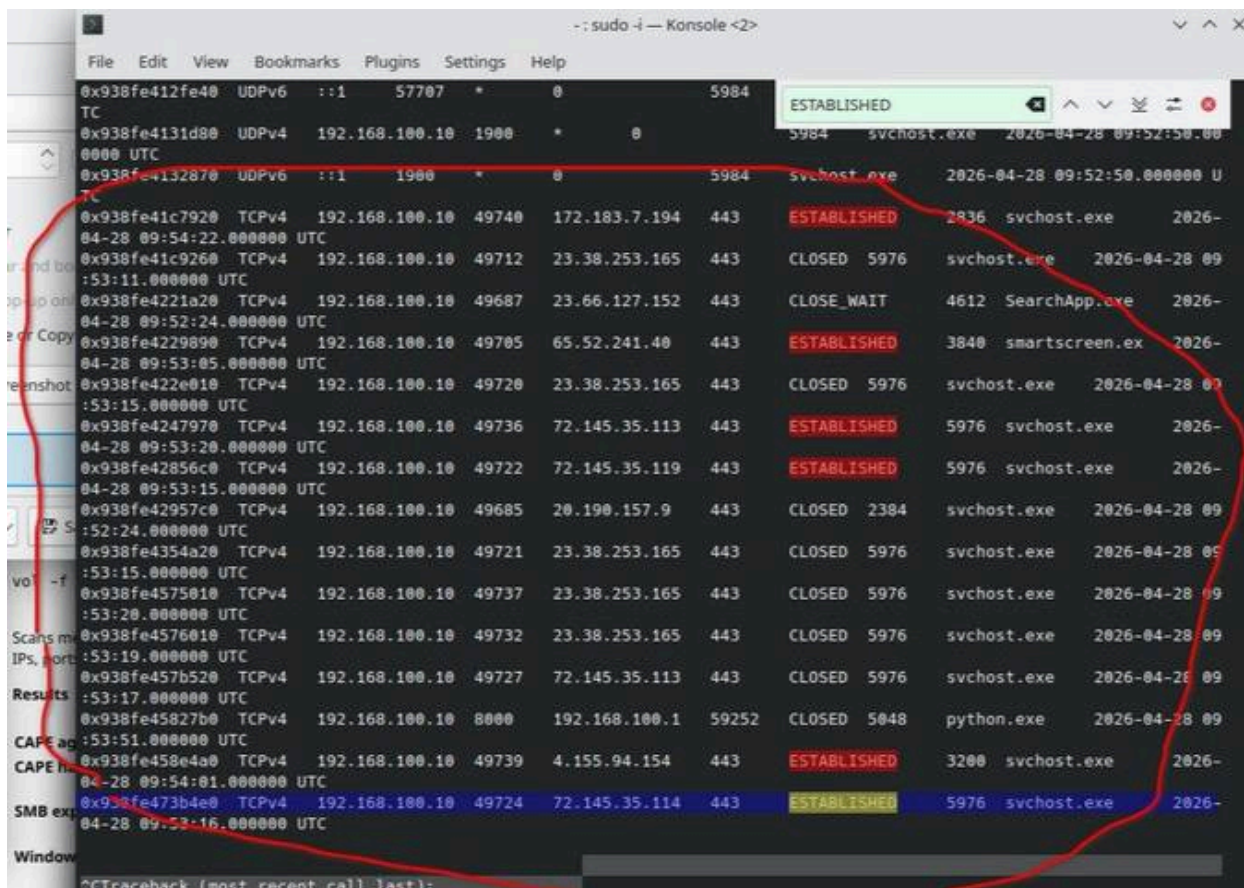


Figure 4.11 — netstat: CAPE host connection (part 2)

Figure 4.11 — *CLOSED* entry: 192.168.100.1:59252 → 192.168.100.10:8000 — CAPE host connected to agent during task execution. ESTABLISHED connections from svchost.exe to Akamai CDN and Microsoft — Windows Update and telemetry traffic*

Findings:

Connection	Owner	Verdict
LISTENING 0.0.0.0:8000	python.exe (5048)	CAPE agent socket
CLOSED 192.168.100.1:59252 → 192.168.100.10:8000	python.exe	CAPE host connecting during analysis
LISTENING 0.0.0.0:445 , 0.0.0.0:139	System (PID 4)	Windows SMB — standard
ESTABLISHED to 23.38.x.x , 72.145.x.x	svchost.exe	Windows Update via Akamai CDN

Connection	Owner	Verdict
ESTABLISHED to 65.52.241.40:443	smartscreen.exe	Microsoft SmartScreen cloud check
CLOSED to 48.192.1.65:443	slui.exe	Windows activation attempt

No malware C2 connections. Every outbound connection belongs to a legitimate Windows process communicating with Microsoft or Akamai infrastructure. Themida exited before any payload network code ran.

Step 6 — Injected Code Detection (windows.malfind)

```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.malfind
```

Finds memory regions that are executable, private (not backed by a file on disk), and contain shellcode-like byte patterns. This is the main plugin for detecting process injection and reflective DLL loading.

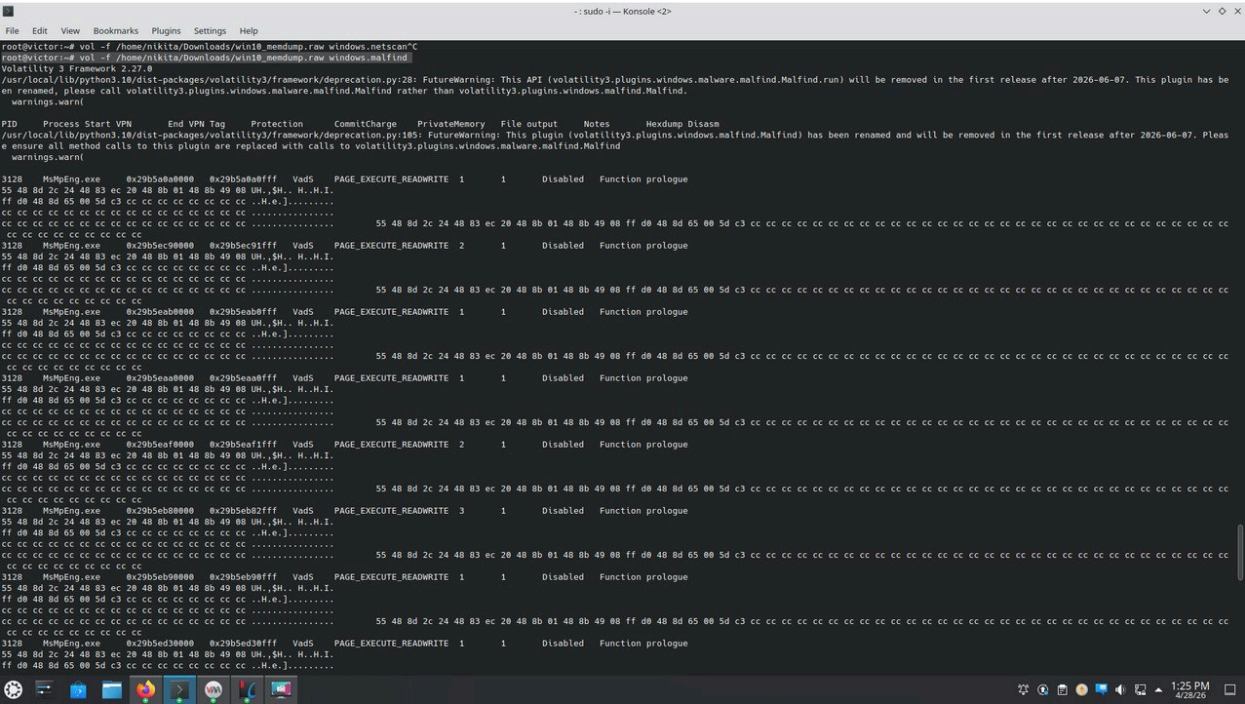


Figure 4.13 — windows.malfind output

Figure 4.13 — malfind results — all hits are in legitimate system processes

Results:

Process	PID	Hits	Verdict
MsMpEng.exe	3128	12	False positive — Windows Defender JIT engine
SearchApp.exe	4612	2	False positive — .NET CLR JIT dispatch stubs
smartscreen.exe	3840	1	False positive — .NET CLR JIT stub
powershell.exe	3480	5	False positive — PowerShell is a .NET app, CLR allocates RWX regions
Malware sample	—	0	No injection artifacts

All 20 hits are explained. The `MsMpEng.exe` pattern is identical across all 12 hits:

```
55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08
ff d0 48 8d 65 00 5d c3 cc cc cc cc cc cc cc cc
```

This is a standard x64 function prologue followed by `INT3` padding (`cc`) — a compiled Defender scan stub, not shellcode. The `cc` padding is what triggers malfind’s heuristic.

Key finding: No MZ headers in private memory of any process. No shellcode. Themida exited cleanly without injecting anything into a host process.

Step 7 — Loaded DLLs (`windows.dlllist`)

```
vol -f /home/nikita/Downloads/win10_memdump.raw windows.dlllist --pid 5048
```

Lists all DLLs loaded into the CAPE agent process from the PEB loader list. Used to verify no unexpected modules were injected into the agent.

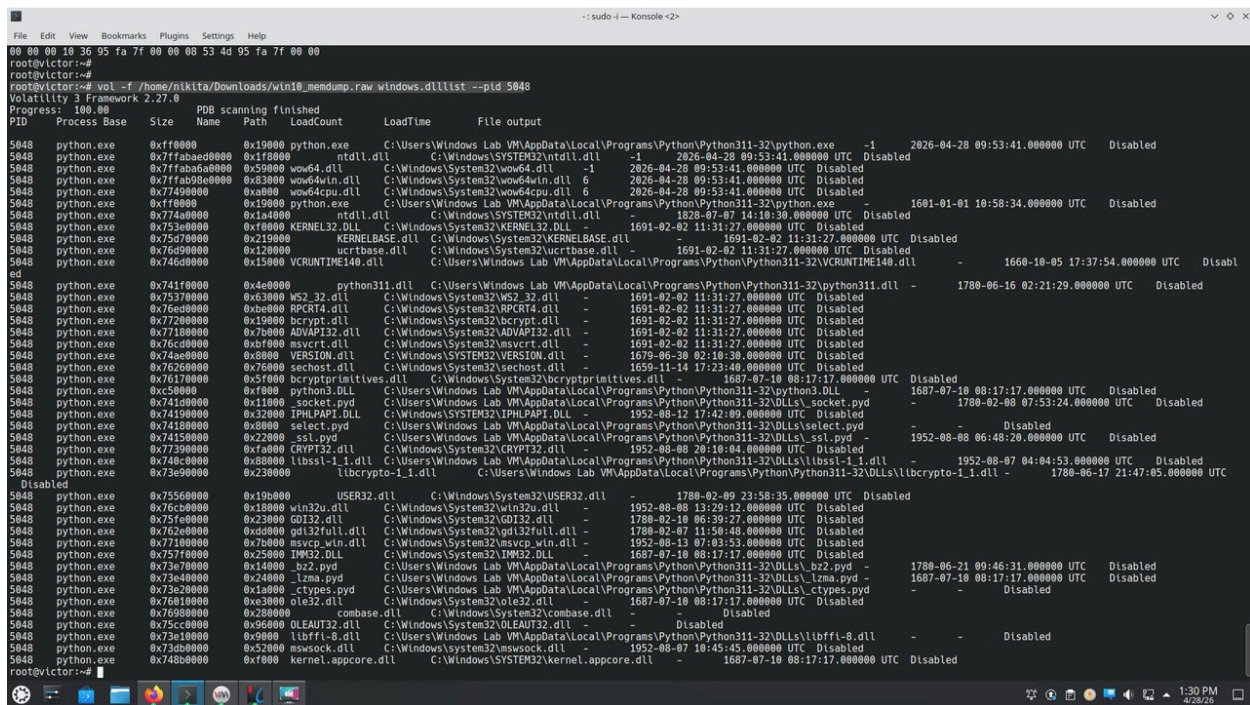


Figure 4.14 — windows.dllexport output for PID 5048

Figure 4.14 — All DLLs loaded into python.exe (5048)

Key modules loaded:

DLL	Purpose
python311.dll	Python 3.11 runtime
_socket.pyd	Network socket module
_ssl.pyd + libssl-1_1.dll	TLS/SSL — agent encrypts result traffic
_ctypes.pyd + libffi-8.dll	Direct Windows API calls via ctypes
IPHLPAPI.DLL	Network interface queries
wow64.dll / wow64win.dll	WoW64 bridge — 32-bit process on 64-bit OS

Two anomalies that are actually expected:

- `ntdll.dll` appears twice at different base addresses — normal for WoW64 processes (both 32-bit and 64-bit `ntdll` are mapped)
- Many DLLs show LoadTime of `1601-01-01` — uninitialized PEB timestamp field, common in WoW64

No unexpected DLLs. No modules loaded from temp paths or unusual locations.

Volatility Summary

Plugin	Key Finding
<code>windows.info</code>	Windows 10 Build 19041, x64, dump valid
<code>windows.pslist</code>	100+ processes, all legitimate; agent visible at PID 5048
<code>windows.pstree</code>	explorer → powershell → python.exe chain confirmed, no anomalies
<code>windows.cmdline</code>	Agent launched as <code>.\agent.py</code> interactively; no encoded PowerShell
<code>windows.netscan</code>	Agent socket on :8000; CAPE handshake visible; all external traffic is Windows telemetry
<code>windows.malfind</code>	20 hits, all JIT false positives from Defender and .NET CLR
<code>windows.dlllist</code>	All agent DLLs expected; no injected modules

Overall: The memory dump confirms that no malware activity reached the injection or execution stage in any of the four analysis runs. Themida's protection exited the process cleanly before any payload ran.

3.5 Online Sandbox Comparison

Two external platforms were evaluated alongside CAPEv2: **ANY.RUN** and **Hybrid Analysis** (powered by CrowdStrike Falcon Sandbox).

ANY.RUN

ANY.RUN is an interactive sandbox — unlike automated tools, the analyst controls the mouse and keyboard inside the running VM through a browser in real time.

Key features:

- Live mouse and keyboard input during analysis — this directly defeats `mouse_movement_detect` and `antisandbox_foregroundwindows`
- MITRE ATT&CK technique mapping per process
- Full PCAP with DNS and TLS inspection
- Suricata IDS rules and YARA on live traffic
- Configurable OS locale, screen resolution, installed software
- Limitation: cloud-based — some APT malware checks for cloud IP ranges and refuses to run

Free tier: public submissions only, 60-second timeout.

Hybrid Analysis (Falcon Sandbox)

Fully automated, backed by CrowdStrike's global threat intelligence.

Key features:

- Cross-references results against CrowdStrike's global IOC database
- Full ATT&CK matrix mapping
- Deep static string scan even before execution — if the payload matches a known family, a verdict is returned without needing to run it
- Limitation: no live interaction — same evasion weakness as CAPEv2 for Themida-class samples

Comparison

Feature	CAPEv2 (local)	ANY.RUN	Hybrid Analysis
Behavioral data on Sample 2	Zero — Themida evasion	Possibly partial — human interaction bypasses some checks	Likely zero — fully automated
Evasion defeat	Manual VM hardening	Built-in interaction simulation	Not available
Nested PE extraction	Yes — auto-extracted Sample 2	No	No
Network capture	Full PCAP on host	Full PCAP + TLS decode	Full PCAP
Memory analysis	Manual via Volatility	Not exposed	Not exposed
Sample privacy	Fully local	Sample uploaded to cloud	Sample uploaded to cloud
VM customization	Full control — SMBIOS, CPUID, MAC	Vendor-controlled	Minimal
Cost	Free (self-hosted)	Paid for advanced features	Paid for private submissions

Conclusions

- **ANY.RUN** is the best option when evasion is the problem. Real mouse input can bypass checks that defeat all automated tools. For Sample 2, an analyst running it interactively on ANY.RUN might have gotten behavioral data.

- **Hybrid Analysis** is strong for known malware families — the threat intel database can return a verdict from static patterns alone.
- **CAPEv2** is the only platform that gave us the nested PE extraction, full memory access via Volatility, and complete control over the VM environment. For deep research and sensitive samples, local is the right choice.

The three approaches are complementary. ANY.RUN for evasion-resistant samples. Hybrid Analysis for quick triage against known families. CAPEv2 for deep, private, configurable analysis.

Task 4 — Static Analysis

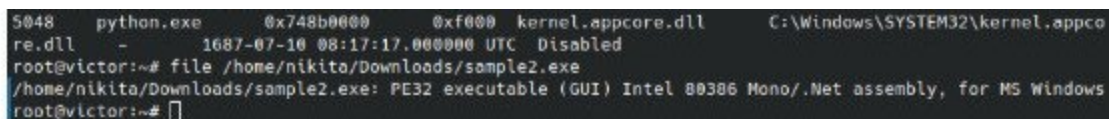
Since dynamic analysis produced zero behavioral data across four attempts, static analysis was the only remaining option. All tools were run on the host. Target:

`0ba17c89...` (Sample 2, the Themida-packed nested PE).

Tools used: `file`, `pefile` (Python), `strings`, `binwalk`, `rabin2`, `r2` (radare2).

Step 1 — File Identification

```
file /home/nikita/Downloads/sample2.exe
```



```
5048 python.exe @x748b0000 @xf000 kernel.appcore.dll C:\Windows\SYSTEM32\kernel.appco
re.dll - 1687-07-10 08:17:17.000000 UTC Disabled
root@victor:~# file /home/nikita/Downloads/sample2.exe
/home/nikita/Downloads/sample2.exe: PE32 executable (GUI) Intel 80386 Mono/.Net assembly, for MS Windows
root@victor:~#
```

Figure 5.1 — file command output

Figure 5.1 — `file` output: PE32 executable, GUI, Mono/.NET assembly

Output: `PE32 executable (GUI) Intel 80386 Mono/.Net assembly, for MS Windows`

- 32-bit Windows GUI executable
- `.Net assembly` in the output means the binary contains .NET metadata — even though it is packed, the PE header still carries the .NET marker

- GUI subsystem — no console window, designed to run as a desktop application
-

Step 2 — PE Header and Section Entropy

```
python3 /home/nikita/Downloads/pe_analysis.py
```

The script reads the PE headers and calculates Shannon entropy for each section.


```

-: sudo -i — Konsole <2>
File Edit View Bookmarks Plugins Settings Help
0x{s.Misc_VirtualSize:08x} 0x{s.SizeOfRawData:08x} {ent:8.4f}')
"
File "<string>", line 2
import pefile, math
IndentationError: unexpected indent
root@victor:~# cat /home/nikita/Downloads/pe_analysis.py
import pefile, math

pe = pefile.PE('/home/nikita/Downloads/sample2.exe')
print(f'Entrypoint: 0x{pe.OPTIONAL_HEADER.AddressOfEntryPoint:08x}')
print(f'ImageBase: 0x{pe.OPTIONAL_HEADER.ImageBase:08x}')
print(f'Timestamp: {pe.FILE_HEADER.TimeDateStamp} (0x{pe.FILE_HEADER.TimeDateStamp:08x})')
print(f'Subsystem: {pe.OPTIONAL_HEADER.Subsystem}')
print(f'Compiled: {pefile.PE.__module__}')
print()
print(f'{"Section":<12} {"VirtAddr":>12} {"VirtSize":>12} {"RawSize":>10} {"Entropy":>9}')
print('-'*62)
for s in pe.sections:
    data = s.get_data()
    if len(data) > 0:
        freq = [data.count(bytes([i])) for i in range(256)]
        ent = -sum((f/len(data))*math.log2(f/len(data)) for f in freq if f > 0)
    else:
        ent = 0
    name = s.Name.decode(errors='replace').strip('\x00')
    print(f'{name:<12} 0x{s.VirtualAddress:08x} 0x{s.Misc_VirtualSize:08x} 0x{s.SizeOfRawData:08x}
{ent:8.4f}')

print()
print('=== IMPORTS ===')
if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
    for lib in pe.DIRECTORY_ENTRY_IMPORT:
        print(f'\n{lib.dll.decode()}')
        for imp in lib.imports[:15]:
            name = imp.name.decode() if imp.name else f'ord_{imp.ordinal}'
            print(f' {name}')
else:
    print('No standard imports (packed/protected)')
root@victor:~# python3 /home/nikita/Downloads/pe_analysis.py
Entrypoint: 0x0010f93e
ImageBase: 0x00400000
Timestamp: 2357873377 (0x8c8a4ae1)
Subsystem: 2
Compiled: pefile

Section          VirtAddr      VirtSize      RawSize      Entropy
-----
.text            0x00002000    0x0010d944    0x0010da00    7.8711
.rsrc            0x00110000    0x00000678    0x00000800    3.5972
.reloc           0x00112000    0x0000000c    0x00000200    0.1019

=== IMPORTS ===

mscoree.dll
_CorExeMain
root@victor:~# python3 /home/nikita/Downloads/pe_analysis.py

```

Figure 5.2 — PE header and section entropy output

Figure 5.2 — pe_analysis.py output showing headers, sections, and entropy values

Header fields:

Field	Value	Meaning
Entrypoint	0x0010f93e	Single jump stub — no real code
ImageBase	0x00400000	Standard Windows default
Timestamp	0x8c8a4ae1 = Sep 2044	Future date — deliberate timestomping
Subsystem	2 = Windows GUI	Desktop application, no console

Section entropy:

Section	Size	Entropy	Verdict
.text	1.05 MB	7.87 / 8.0	Themida-encrypted — near-maximum
.rsrc	2 KB	3.60	Normal resource data
.reloc	12 bytes	0.10	Nearly empty — packed binary artifact

Normal compiled code sits around 4.5–6.5 entropy. 7.87 is the forensic definition of encryption or compression. The entire code section is a ciphertext blob that cannot be disassembled in static form.

Imports:

```
mscoree.dll → _CorExeMain (only import)
```

One function. Themida resolves all its own API calls dynamically at runtime — nothing appears in the import table because the real import logic is inside the encrypted section.

Step 3 — String Extraction

```
strings -n 6 /home/nikita/Downloads/sample2.exe \
| grep -E "[A-Za-z0-9 _.\-\\/\:]{6,}$" | sort -u
```

Despite 7.87 entropy in the code section, the .NET metadata region (below offset 0x1E000) is not encrypted and leaks readable strings.

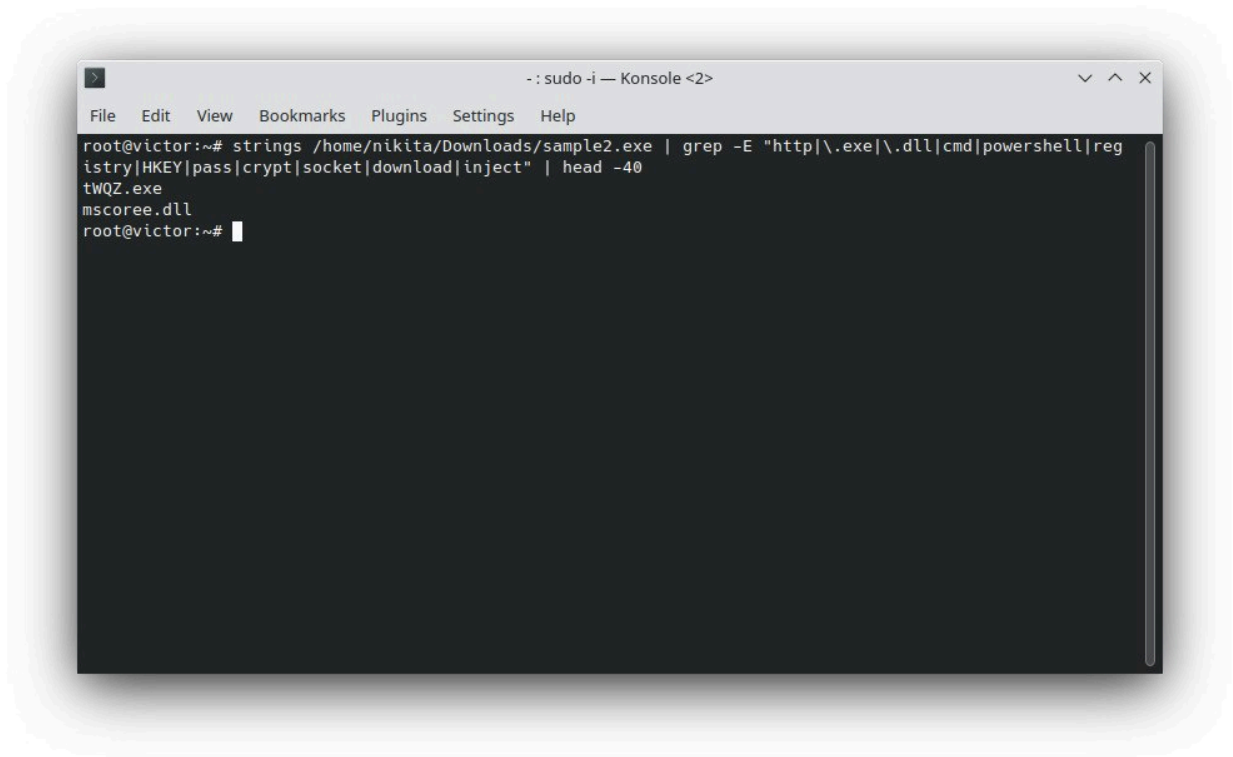


Figure 5.3 — String extraction overview

Figure 5.3 — Filtered strings output showing multiple readable namespaces from .NET metadata

The strings revealed the true identity of the payload.

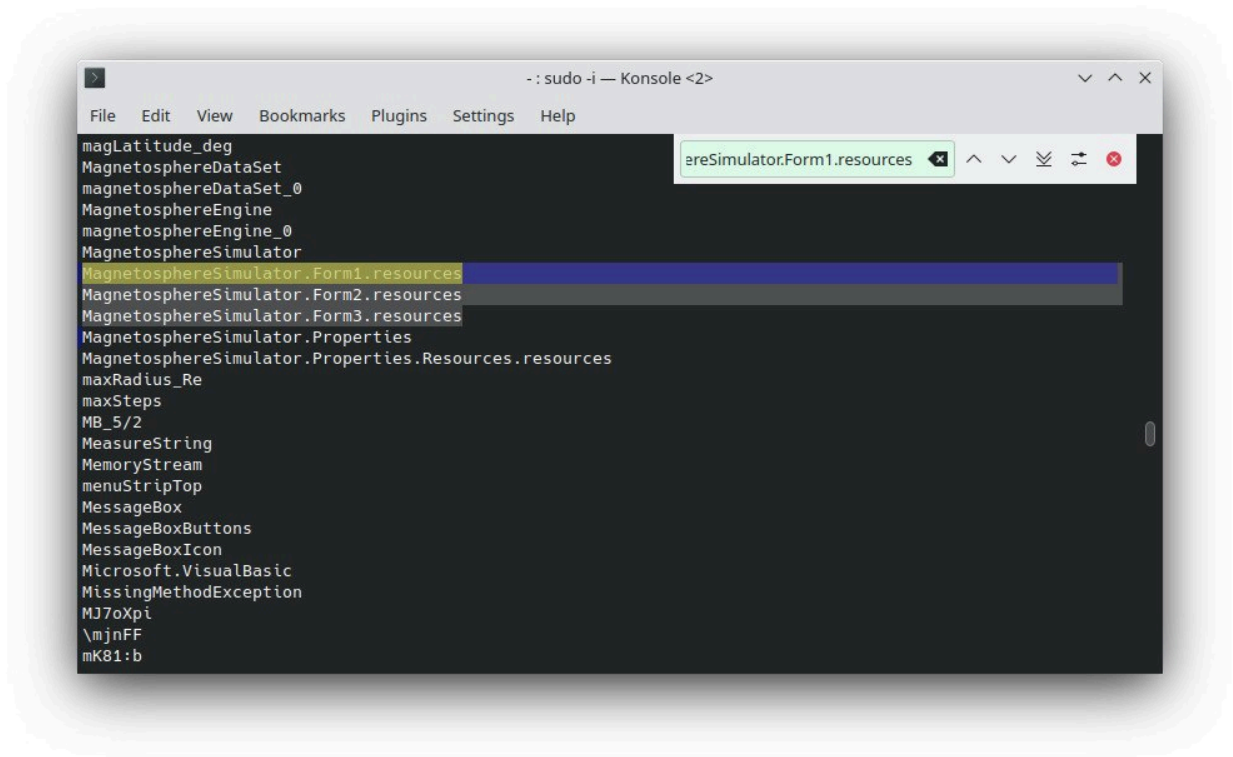


Figure 5.4 — Strings: MagnetosphereSimulator namespace

Figure 5.4 — `MagnetosphereSimulator.Form*.resources` — application name and UI form namespaces

Application identity:

String	Meaning
<code>MagnetosphereSimulator</code>	Application name
<code>Orbital Plasma Labs</code>	Author / company
<code>1.0.5.1</code>	Application version
<code>v4.0.30319</code>	.NET Framework 4.0 runtime

Scientific functionality visible in the MSIL metadata:

Method / Namespace	Purpose
<code>ComputeDipoleField</code>	Earth magnetic dipole model
<code>ComputeGyroFrequency</code>	Particle gyrofrequency calculation
<code>ComputeLShell</code>	L-shell (magnetic latitude)

Method / Namespace	Purpose
<code>ComputeDriftPeriod</code>	Van Allen belt drift period
<code>ComputeBowShockRe</code>	Bow shock standoff distance
<code>ComputeMagnetopauseStandoffRe</code>	Magnetopause boundary
<code>DrawBowShock</code> , <code>DrawMagnetopause</code> , <code>DrawBelts</code>	Visualization engine
<code>DrawSolarWindArrows</code> , <code>DrawTrajectories</code>	Particle trajectory rendering
Particle types: <code>Proton</code> , <code>Electron</code> , <code>Helium</code> , <code>Oxygen</code>	Multi-species simulation

This is a full-featured GUI physics application. Not malware.

Additional key strings:

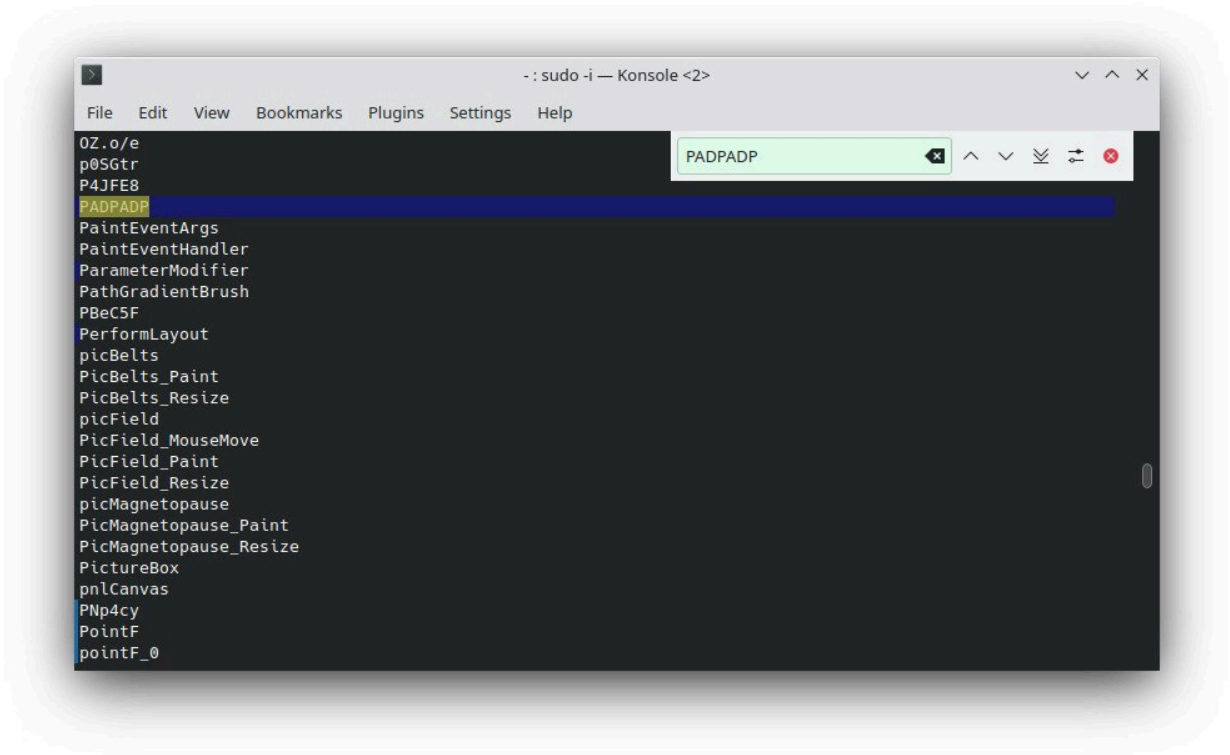


Figure 5.5 — Strings: PADPADP marker

Figure 5.5 — `PADPADP` found in the binary — a known Themida internal padding marker

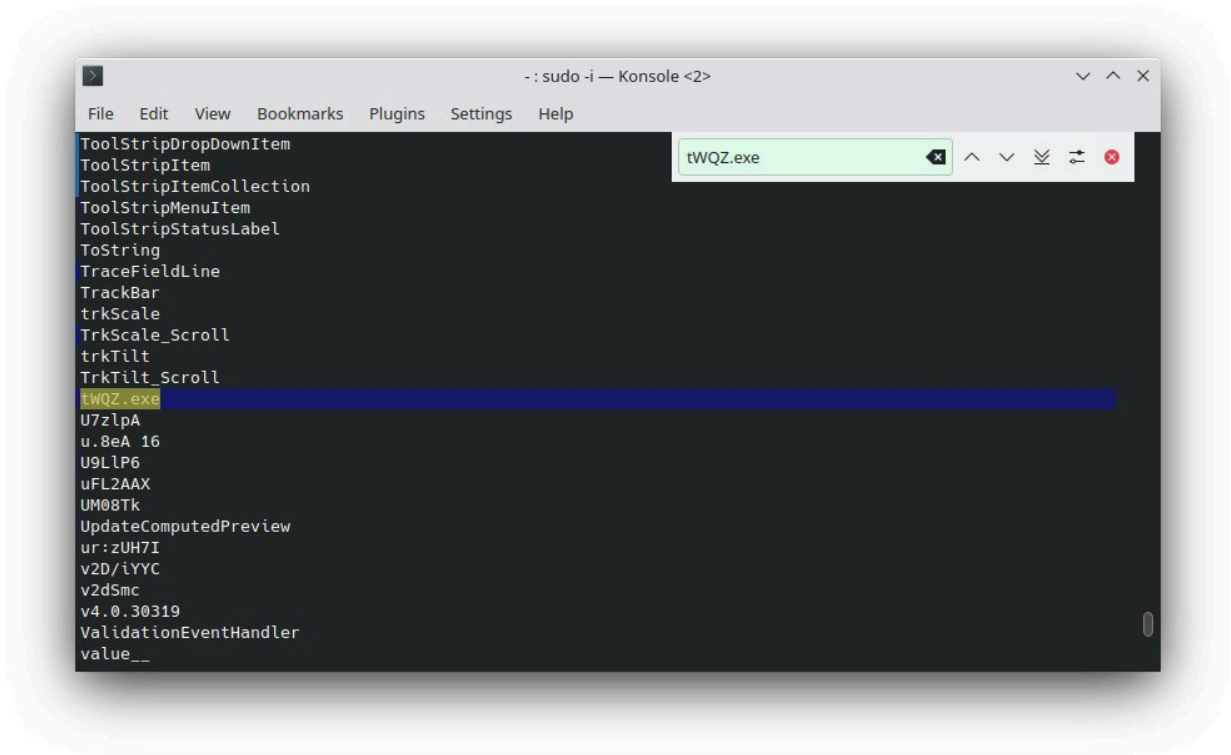


Figure 5.6 — Strings: tWQZ.exe reference

Figure 5.6 — `tWQZ.exe` — randomly generated filename, likely the original name of the outer dropper

String	Significance
<code>PADPADP</code>	Themida internal padding marker — definitive Themida confirmation
<code>tWQZ.exe</code>	Random filename — likely the outer dropper’s original name

Step 4 — Entropy Map

```
binwalk -E /home/nikita/Downloads/sample2.exe
```

Produces a visual entropy plot showing how entropy changes across the file.

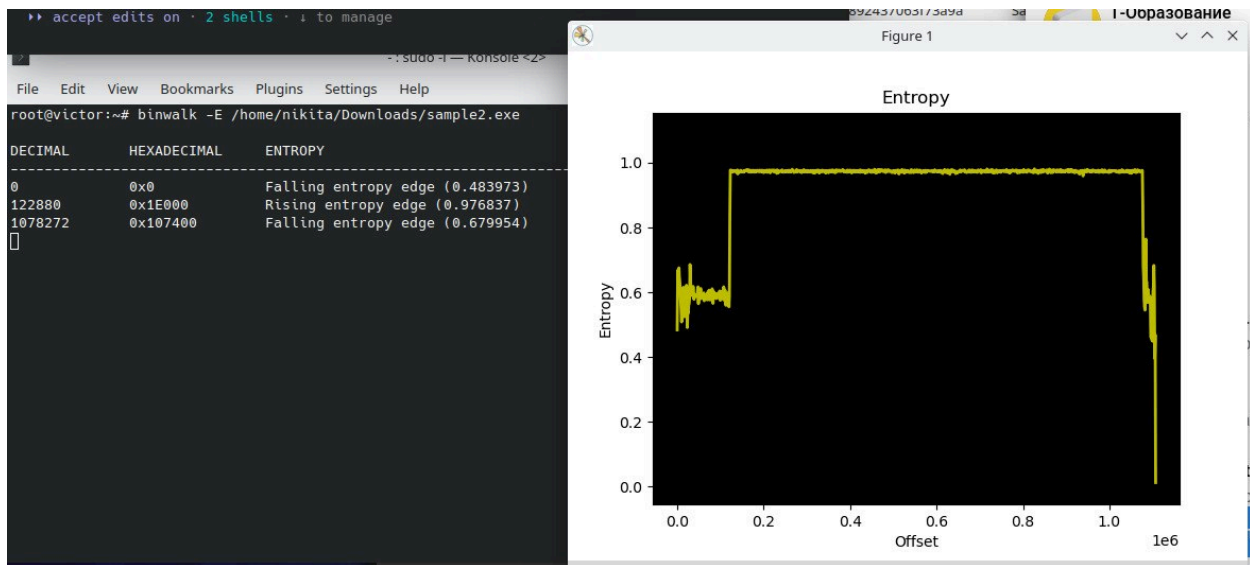


Figure 5.7 — binwalk -E entropy map

Figure 5.7 — Sharp entropy rise at offset 0x1E000 marks the start of Themida's encrypted section

Offset range	Entropy	Region
0x0 → 0x1E000	~0.48	PE headers and .NET metadata — readable
0x1E000 → 0x107400	~0.98	Themida-encrypted <code>.text</code> — 955 KB of cipher
0x107400 → EOF	~0.68	<code>.rsrc</code> resources — partially structured

The sharp rise at `0x1E000` is the exact byte where Themida's encryption starts. 97.7% entropy is statistically indistinguishable from random data.

Step 5 — PE Metadata

```
rabin2 -I /home/nikita/Downloads/sample2.exe
```

```
> - : sudo -i — Konsole <2>
File Edit View Bookmarks Plugins Settings Help
root@victor:~# rabin2 -I /home/nikita/Downloads/sample2.exe
arch      x86
baddr     0x400000
binsz     1107456
bintype   pe
bits      32
canary    true
injprot   false
retguard  false
class     PE32
cmp.csum  0x00011e0c4
compiled  Mon Sep 19 04:49:37 2044
crypto    false
endian    little
havecode  true
hdr.csum  0x00000000
laddr     0x0
lang      cil
linenum   false
lsyms     false
machine   i386
nx        true
os        windows
overlay   false
cc        cdecl
pic       true
relocs    false
signed    false
sanitize  false
static    false
stripped  false
subsys    Windows GUI
va        true
root@victor:~#
```

Figure 5.8 — rabin2 -I output

Figure 5.8 — rabin2 PE metadata: compiled date, language, signing status, checksum fields

Key fields:

Field	Value	Meaning
<code>compiled</code>	Mon Sep 19 04:49:37 2044	Timestomping — confirmed independently by rabin2
<code>lang</code>	<code>cil</code>	.NET CIL bytecode
<code>signed</code>	<code>false</code>	No valid code signature — cracked or modified binary
<code>hdr.csum</code>	<code>0x00000000</code>	PE checksum zeroed
<code>cmp.csum</code>	<code>0x0011e0c4</code>	Calculated real checksum

The mismatch between `hdr.csum` (0) and `cmp.csum` (0x0011e0c4) is the forensic fingerprint of a patched binary. When software is cracked, patching tools typically zero the PE checksum rather than recalculate it correctly.

`canary=true` and `nx=true` — these came from the original legitimate build and were preserved through the crack process.

Step 6 — Import Table

```
rabin2 -i /home/nikita/Downloads/sample2.exe
```

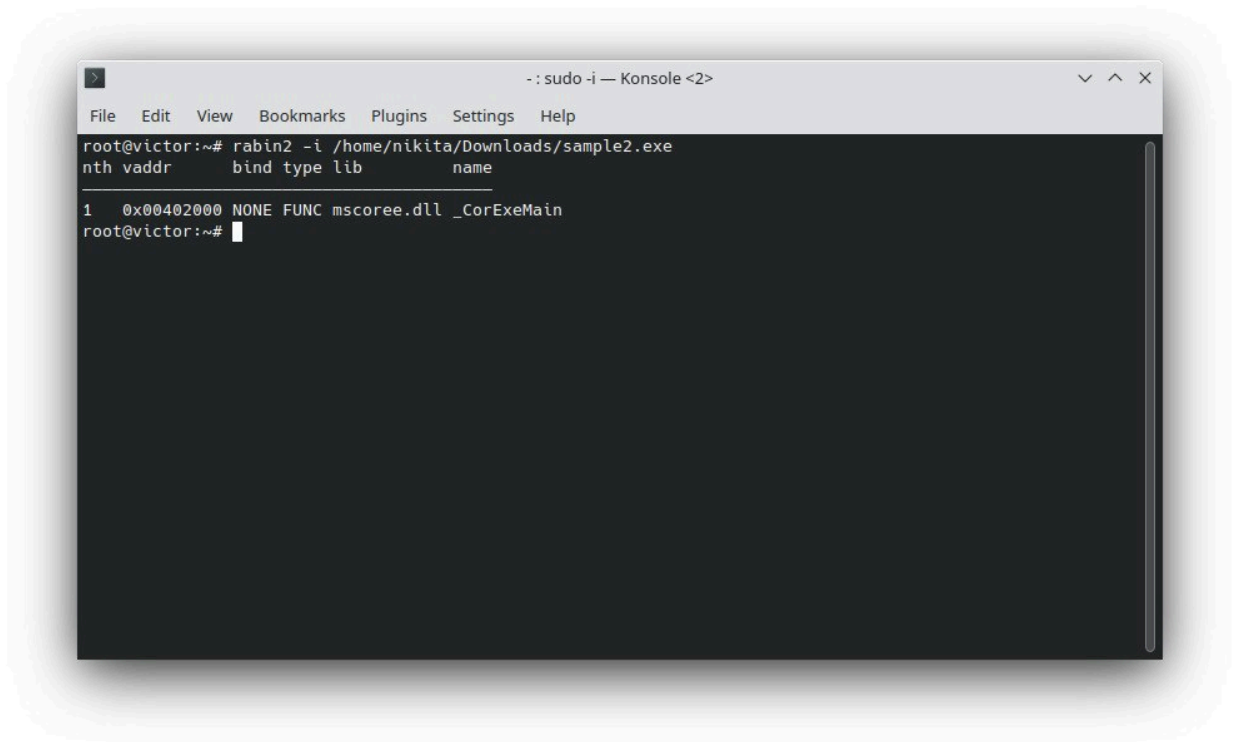


Figure 5.9 — `rabin2 -i` import table

- Figure 5.9 — Single import: `mscoree.dll` → `_CorExeMain*`

Output:

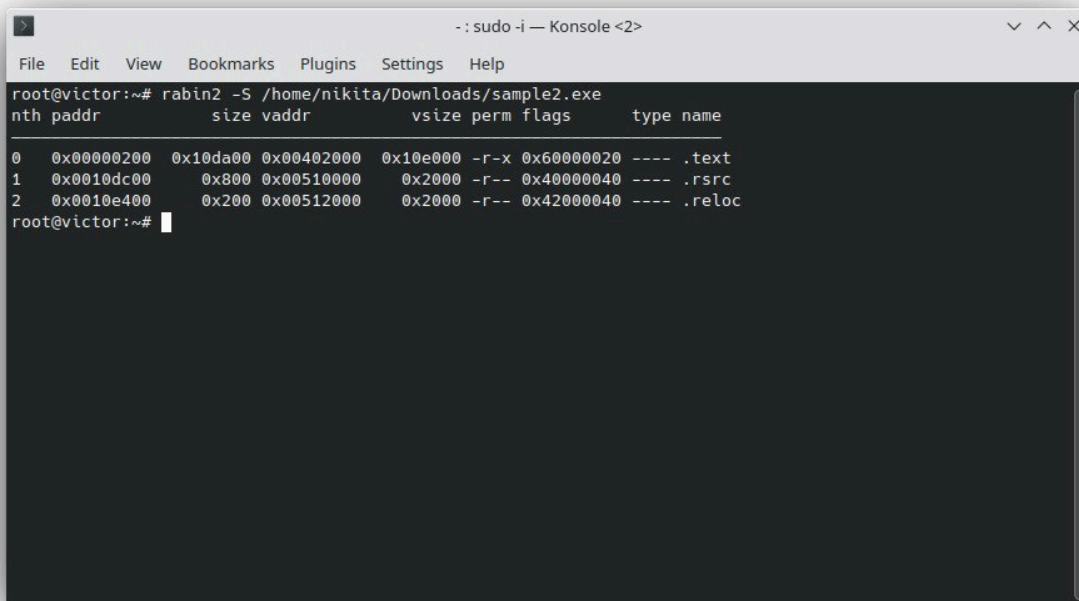
```
1  0x00402000  mscoree.dll  _CorExeMain
```

One import. All of Themida's API calls are resolved internally through its own hash-based lookup at runtime. The actual IAT that matters is encrypted inside

`.text`.

Step 7 — Section Map

```
rabin2 -S /home/nikita/Downloads/sample2.exe
```



```
root@victor:~# rabin2 -S /home/nikita/Downloads/sample2.exe
nth paddr      size vaddr      vsize perm flags      type name
---
0  0x00000200 0x10da00 0x00402000 0x10e000 -r-x 0x60000020 ---- .text
1  0x0010dc00   0x800 0x00510000  0x2000 -r-- 0x40000040 ---- .rsrc
2  0x0010e400   0x200 0x00512000  0x2000 -r-- 0x42000040 ---- .reloc
root@victor:~#
```

Figure 5.10 — rabin2 -S section map

Figure 5.10 — Three sections: .text (1.05 MB), .rsrc (2 KB), .reloc (12 bytes)

.text	0x00402000	size=0x10da00	vsize=0x10e000	-r-x
.rsrc	0x00510000	size=0x800	vsize=0x2000	-r--
.reloc	0x00512000	size=0x200	vsize=0x2000	-r--

`.text` is `-r-x` (no write) on disk. At runtime, Themida calls `VirtualProtect` to temporarily mark it writable, decrypts the real payload in-place, then restores `-r-x`. This is the standard unpack-in-place technique.

Step 8 — Entry Point Disassembly

```
r2 -q -c 'pd 30' /home/nikita/Downloads/sample2.exe 2>/dev/nu
ll
```

```

-: sudo -i — Konsole <2>
File Edit View Bookmarks Plugins Settings Help
root@victor:~# r2 -q -c 'pd 30' /home/nikita/Downloads/sample2.exe 2>/dev/null
;-- entry0:
0x0050f93e ff2500204000 jmp dword [sym.imp.mscoree.dll__CorExeMain] ; 0x402000 ; "\xf9\x10"
0x0050f944 0000 add byte [eax], al
0x0050f946 0000 add byte [eax], al
0x0050f948 0000 add byte [eax], al
0x0050f94a 0000 add byte [eax], al
0x0050f94c 0000 add byte [eax], al
0x0050f94e 0000 add byte [eax], al
0x0050f950 0000 add byte [eax], al
0x0050f952 0000 add byte [eax], al
0x0050f954 0000 add byte [eax], al
0x0050f956 0000 add byte [eax], al
0x0050f958 0000 add byte [eax], al
0x0050f95a 0000 add byte [eax], al
0x0050f95c 0000 add byte [eax], al
0x0050f95e 0000 add byte [eax], al
0x0050f960 0000 add byte [eax], al
0x0050f962 0000 add byte [eax], al
0x0050f964 0000 add byte [eax], al
0x0050f966 0000 add byte [eax], al
0x0050f968 0000 add byte [eax], al
0x0050f96a 0000 add byte [eax], al
0x0050f96c 0000 add byte [eax], al
0x0050f96e 0000 add byte [eax], al
0x0050f970 0000 add byte [eax], al
0x0050f972 0000 add byte [eax], al
0x0050f974 0000 add byte [eax], al
0x0050f976 0000 add byte [eax], al
0x0050f978 0000 add byte [eax], al
0x0050f97a 0000 add byte [eax], al
0x0050f97c 0000 add byte [eax], al
root@victor:~#

```

Figure 5.11 — r2 entry point disassembly

- Figure 5.11 — Entry point: single jmp to _CorExeMain, followed by encrypted null bytes misread as opcodes*

Output:

```

;-- entry0:
0x0050f93e ff2500204000 jmp dword [sym.imp.mscoree.dll__CorExeMain]
0x0050f944 0000 add byte [eax], al ; encrypted bytes
0x0050f946 0000 add byte [eax], al ; misread a

```



```
s instructions
...
```

One instruction: jump to the .NET CLR entry point. Everything after is `0x00 0x00` from the encrypted section being misread as x86 opcodes. Themida's decryption loop runs entirely inside the encrypted section at runtime — there is nothing to statically disassemble.

Static Analysis Conclusion — True Sample Identity

Static analysis identified the payload as **MagnetosphereSimulator v1.0.5.1 by Orbital Plasma Labs** — a cracked commercial scientific application for Earth magnetosphere and Van Allen belt simulation. It is not conventional malware.

Themida was used as **DRM (Digital Rights Management)** to protect the software license, not to conceal malicious behavior. This explains every anomaly from dynamic analysis:

Observation	Explanation
13 anti-sandbox techniques	Themida DRM checking for VM to prevent license bypass
No C2, no destructive behavior	There is no malicious payload — Themida was protecting legitimate software
Timestomping	Cracked software obfuscating its origin
<code>hdr.csum = 0x00000000</code>	Patching tool zeroed the checksum during the crack process
<code>signed = false</code>	Code signature invalidated by patching
Refused to run in QEMU across 4 attempts	Themida DRM is specifically designed to detect VMs

Dynamic vs Static — Which Worked Better?

	Dynamic (CAPEv2)	Static (CLI tools)
Found	Outer dropper structure, nested PE extraction, 13	True application identity, Themida confirmation, timestomping, patched

	Dynamic (CAPEv2)	Static (CLI tools)
	evasion signatures	checksum, version, author
Missed	Everything about the inner payload	Runtime behavior, decryption keys, actual execution flow
Strength	Real behavior — network IoCs, process chains, registry changes	No evasion possible — file exists on disk, metadata is always readable
Weakness	Themida/DRM defeats all automated analysis	Encrypted sections cannot be disassembled without unpacking

For this specific sample, static analysis was far more productive. The sandbox ran the binary 4 times and captured nothing. One `strings` command identified the application name, author, and version in seconds.

That said, the two approaches are complementary. For malware that actually executes — ransomware, RATs, stealers — dynamic analysis produces IoCs (network addresses, dropped files, registry keys, process chains) that static analysis could never find. For packed or DRM-protected software, static analysis of the unencrypted metadata gives answers that no amount of sandbox tweaking can produce.

Appendix — Commands Reference

```
# CAPE services
sudo systemctl restart cape.service cape-processor.service
sudo systemctl status cape.service --no-pager

# VM management
virsh list --all
virsh domstate win10
virsh snapshot-list win10
virsh snapshot-info win10 snapshot1
virsh snapshot-create-as --domain win10 --name snapshot1 --atomic
```

```

# Verify agent
nc -z -w2 192.168.100.10 8000 && echo "AGENT OK"

# Submit sample
cd /opt/CAPEv2
sudo -u cape .cache/pypoetry/virtualenvs/capev2-t2x27zRb-py3.
10/bin/python \
    utils/submit.py /path/to/sample.exe --timeout 240 --enforce
-timeout

# Memory dump
virsh dump win10 /home/nikita/Downloads/win10_memdump.raw --m
emory-only

# Volatility
vol -f /home/nikita/Downloads/win10_memdump.raw windows.info
vol -f /home/nikita/Downloads/win10_memdump.raw windows.pslis
t
vol -f /home/nikita/Downloads/win10_memdump.raw windows.pstre
e
vol -f /home/nikita/Downloads/win10_memdump.raw windows.cmdli
ne
vol -f /home/nikita/Downloads/win10_memdump.raw windows.netsc
an
vol -f /home/nikita/Downloads/win10_memdump.raw windows.malfi
nd
vol -f /home/nikita/Downloads/win10_memdump.raw windows.dllli
st --pid 5048

# Static analysis
file /home/nikita/Downloads/sample2.exe
python3 /home/nikita/Downloads/pe_analysis.py
strings -n 6 /home/nikita/Downloads/sample2.exe | grep -E "^
[A-Za-z0-9 _.\-\\/\:]{6,}$" | sort -u
binwalk -E /home/nikita/Downloads/sample2.exe
rabin2 -I /home/nikita/Downloads/sample2.exe

```

```
rabin2 -i /home/nikita/Downloads/sample2.exe  
rabin2 -S /home/nikita/Downloads/sample2.exe  
r2 -q -c 'pd 30' /home/nikita/Downloads/sample2.exe 2>/dev/nu  
ll
```